

学校代码: 10255  
学 号: 2222747  
中图分类法: TP311



## 专业学位硕士学位论文

# 非独立同分布场景下的联邦学习鲁棒性的优化研究

学位申请人: 谈轩

指导教师: 石秀金

一级学科: 电子信息

研究方向: 联邦学习

所在学院: 计算机科学与技术学院

提交日期: 2025 年 4 月

University Code: 10255

Student ID: 2222747

CLC Index: TP311



## PROFESSIONAL MASTER DISSERTATION

# Optimization Research on the Robustness of Federated Learning in Non-IID scenarios

**Candidate:** Tan Xuan

**Supervisor:** Shi Xiujin

**Major:** Electronic Information

**Research Direction:** Federated Learning

**Affiliation:** College of Computer Science and Technology

**Date of Submit:** Apr, 2025

# 硕士学位论文答辩委员会名单

| 姓名  | 职称    | 单位            | 备注 |
|-----|-------|---------------|----|
| 夏小玲 | 教授    | 东华大学          | 主席 |
| 吴国文 | 副教授   | 东华大学          | 委员 |
| 王绍宇 | 副教授   | 东华大学          | 委员 |
| 于守健 | 副教授   | 东华大学          | 委员 |
| 丁宏锴 | 高级工程师 | 亚太计算机信息系统有限公司 | 委员 |
| 张红  | 高级实验师 | 东华大学          | 秘书 |
|     |       |               |    |

答 辩 人：谈轩

答辩地点：东华大学一号学院楼 259

答辩日期：2025.05.20

# 非独立同分布场景下的联邦学习鲁棒性的优化研究

## 摘要

在联邦学习 (FL, Federated Learning) 中, 拜占庭攻击是一个难以避免的安全挑战。此前部分联邦学习的防御方法声称自己具备鲁棒性, 这些方法通常依赖于在独立同分布 (IID) 条件下, 利用统计筛选离群异常更新来降低攻击的影响。然而, 在联邦学习的实际场景中, 数据的非独立同分布 (Non-IID) 特性使得基于统计规律的筛选方法的准确性难以得到有效保障。为了提高联邦学习模型的鲁棒性, 论文提出了以下三点改进:

(1) 为了提高联邦学习 Non-IID 场景下恶意节点的识别精度, 论文提出一种基于模型更新一致性原理的客户端评分算法。该机制通过 L-BFGS 算法结合客户端历史轮次的梯度变化以较低算力代价计算 Hessian 矩阵, 并将客户端的本地模型训练趋势与 Hessian 矩阵的迭代过程相关联。算法依据客户端最近  $N$  轮梯度预测值和实际值的欧几里得距离得出客户端的可疑分数。在 MNIST 和 CIFAR10 数据集上多种场景下的实验的结果表示, 使用该机制的模型在大多数场景下相比其他对比的防御模型在鲁棒性上取得了明显的提升。

(2) 为了更好地处理联邦学习 Non-IID 场景下恶意节点, 论文提出了一种基于可疑分数的动态梯度处理算法。文章把可疑分数大于等于恶意门槛的客户端认为是恶意的, 否则是良性的客户端。为了依据可信度影响模型的学习重心, 论文以良性梯度的可疑分数与恶意门槛的距离为权值, 基于良性梯度构造出一个新的填充梯度, 用它代替被剔除的恶意梯度进行后续聚合, 从而放大聚合算法中的权重效果。论文在多种场景下进行了实验, 该模型在保持原模型优势的同时在一些高强度攻击场景下取得了更好的防御效果。

(3) 为了防御某些专注于篡改少数特定参数的隐蔽的攻击通过控制大量客户端操纵 Median 聚合的情况, 论文提出了一种基于滑动窗口的中值滤波聚合算法。

该算法基于传统 Median 聚合思路并引入了图像和噪声处理中的中值滤波思想，通过滑动窗口引入相邻值从而稀释潜藏的恶意数据，攻击者如果只操控少数特定参数位，就很难主导聚合结果，如果攻击者想让攻击更有效，就必须连带操控周围更多参数，但一旦这么做，就必须牺牲整组参数的结构，更难使模型向其攻击方向训练，也更容易被前文提出的检测方法注意到，所以其能让该类型攻击的难度和成本极大上升。其具体做法是在聚合的时候先按参数进行 L2 归一化，随后考虑参数及其周围窗口内的参数值进行协同处理取中值，最终恢复其度量特性得到聚合结果。文章在 CIFAR10 数据集上基于前文的算法在多种场景下进行实验，验证了该聚合算法在目标场景下的有效性。

**关键词：**联邦学习；鲁棒性；得分机制；拜占庭攻击；非独立同分布。

# Optimization of Robustness of Federated Learning Based on Dynamic Aggregation

## ABSTRACT

In Federated Learning (FL, Federated Learning), Byzantine attacks are an unavoidable security challenge. Previously, some of the defence methods for Federated Learning claimed to be robust, and these methods usually relied on reducing the impact of the attack by using statistical screening of outlier anomalous updates under Independent Identically Distributed (IID) conditions. However, in real-world scenarios of federated learning, the non-Independently Identically Distributed (Non-IID) nature of the data makes it difficult to effectively guarantee the accuracy of statistical law-based screening methods. In order to improve the robustness of the federated learning model, the paper proposes the following three improvements:

(1) In order to improve the recognition accuracy of malicious nodes in the federated learning Non-Independent Identical Distribution (Non-IID) scenario, the thesis proposes a client-side scoring algorithm based on the principle of model update consistency. The mechanism calculates the Hessian matrix by combining the gradient changes of the client's historical rounds with the L-BFGS algorithm at a lower cost of computing power, and correlates the client's local model training trend with the iterative process of the Hessian matrix. The algorithm derives the client's suspicion score based on the Euclidean distance between the predicted and actual values of the client's gradient for the last  $N$  rounds. The results of the experiments in multiple scenarios on the MNIST and CIFAR10 datasets indicate that the model using this mechanism achieves a significant improvement in robustness compared to other compared defence models in most scenarios.

(2) In order to better handle malicious nodes in federated learning non-independent and identically distributed scenarios, the paper proposes a dynamic gradient processing algorithm based on suspicious score. The paper considers clients with suspicious scores greater than or equal to the malicious threshold as malicious, otherwise they are benign clients. In order to influence the learning gravity of the model based on the credibility, the paper takes the distance between the suspicious score of the benign gradient and the malicious threshold as the weight value, constructs a new filled gradient based on the benign gradient, and replaces the rejected malicious gradient with it for the subsequent aggregation, so as to amplify the effect of the weights in the aggregation algorithm. The paper was experimented in a variety of scenarios, and the model achieved better defence results in some high-intensity attack scenarios while maintaining the advantages of the original model.

(3) In order to defend against some hidden attacks that focus on tampering with a few specific parameters by controlling a large number of clients to manipulate the Median

aggregation, the paper proposes a sliding-window based median filter aggregation algorithm. The algorithm is based on the traditional Median aggregation idea and introduces the idea of median filtering in image and noise processing, which introduces neighbouring values through a sliding window to dilute the latent malicious data, and it is difficult for the attacker to dominate the aggregation result if he manipulates only a few specific parameter bits, and if the attacker wants the attack to be more effective, he must manipulate the surrounding parameters along with it, but once he does so, he has to sacrifice the whole set of parameters' structure, making it more difficult to train the model in the direction of its attack, and more likely to be noticed by the previously proposed detection methods, so its can make this type of attack extremely difficult and costly. This is done by first performing L2 normalisation by parameter at the time of aggregation, followed by co-processing to take the median value considering the parameter and the parameter values within its surrounding window, and finally recovering its metric properties to obtain the aggregated results. The article conducts experiments based on the previous algorithm on the CIFAR10 dataset in a variety of scenarios and verifies the effectiveness of the aggregation algorithm in the target scenario.

TAN XUAN

Supervised by: SHI XIUJIN

**KEY WORDS:**Federated Learning, Robustness, Scoring Mechanisms, Byzantine Attacks, Non-Independent Identical Distributions(Non-IID)

## 目 录

|                                          |    |
|------------------------------------------|----|
| 1. 绪论.....                               | 1  |
| 1.1 研究背景与意义.....                         | 1  |
| 1.2 研究现状.....                            | 3  |
| 1.2.1 联邦学习研究.....                        | 3  |
| 1.2.2 联邦学习相关的攻击机制研究.....                 | 4  |
| 1.2.3 联邦学习相关的防御机制研究.....                 | 5  |
| 1.2.4 当前研究工作总结.....                      | 7  |
| 1.3 研究内容.....                            | 8  |
| 1.4 组织结构.....                            | 10 |
| 2. 相关技术介绍 .....                          | 12 |
| 2.1 联邦学习.....                            | 12 |
| 2.1.1 联邦学习概念.....                        | 12 |
| 2.1.2 联邦学习防御介绍.....                      | 13 |
| 2.1.3 联邦学习攻击介绍.....                      | 14 |
| 2.2 Hessian 矩阵 .....                     | 16 |
| 2.2.1 Hessian 矩阵在梯度近似方面的理论 .....         | 16 |
| 2.2.2 L-BFGS 算法在计算 Hessian 矩阵方面的理论 ..... | 17 |
| 2.3 数据集介绍.....                           | 19 |
| 2.4 本章小结.....                            | 20 |
| 3. 基于模型更新一致性的客户端评分算法 .....               | 21 |
| 3.1 算法概述.....                            | 21 |
| 3.2 基于模型更新一致性的客户端评分算法 .....              | 23 |
| 3.2.1 实验模型框架.....                        | 23 |
| 3.2.2 模型更新一致性在实验中的应用.....                | 24 |
| 3.2.3 基于模型一致性的检测机制.....                  | 25 |
| 3.2.4 恶意门槛选取.....                        | 26 |
| 3.3 本章小结.....                            | 28 |
| 4. 基于可疑分数的动态处理梯度算法 .....                 | 30 |
| 4.1 均等权重的缺陷和临界规避行为 .....                 | 30 |
| 4.2 基于可疑分数的动态处理梯度算法 .....                | 31 |
| 4.2.1 算法流程概述.....                        | 31 |
| 4.2.2 算法对可疑分数的影响分析.....                  | 33 |
| 4.3 本章小结.....                            | 35 |
| 5. 基于滑动窗口的中值滤波聚合算法 .....                 | 36 |

|                                |    |
|--------------------------------|----|
| 5.1 传统聚合算法缺陷.....              | 36 |
| 5.2 中值滤波聚合算法.....              | 37 |
| 5.2.1 卷积的参数相关性.....            | 37 |
| 5.2.2 滑动窗口.....                | 38 |
| 5.2.3 算法流程概述.....              | 38 |
| 5.2.4 算法优势分析.....              | 39 |
| 5.2.5 算法潜在缺点分析.....            | 40 |
| 5.3 本章小结.....                  | 40 |
| 6. 实验设置与结果分析 .....             | 42 |
| 6.1 基础实验设置.....                | 42 |
| 6.1.1 防御者的知识与能力.....           | 42 |
| 6.1.2 攻击者的知识与能力.....           | 42 |
| 6.1.3 攻击类型.....                | 42 |
| 6.1.4 聚合方法.....                | 44 |
| 6.1.5 实验模型设置.....              | 44 |
| 6.1.6 联邦学习参数设置.....            | 45 |
| 6.2 可疑分数评分模块与动态处理梯度模块的实验 ..... | 50 |
| 6.2.1 实验分析.....                | 50 |
| 6.2.2 实验小结.....                | 56 |
| 6.3 中值滤波聚合实验.....              | 56 |
| 6.4 本章小结.....                  | 58 |
| 7. 结论与展望 .....                 | 59 |
| 7.1 研究结论.....                  | 59 |
| 7.2 研究展望.....                  | 60 |
| 参考文献.....                      | 61 |

## 图录

|       |                                            |    |
|-------|--------------------------------------------|----|
| 图 1-1 | FLTrust 运作图 .....                          | 6  |
| 图 1-2 | 防御系统架构 .....                               | 10 |
| 图 2-1 | 联邦学习训练流程示意图 .....                          | 13 |
| 图 2-2 | 拜占庭攻击示意图 .....                             | 14 |
| 图 2-3 | 数据和模型中毒攻击示意图 .....                         | 15 |
| 图 3-1 | FLDetector 流程图 .....                       | 21 |
| 图 3-2 | 评分算法系统架构图 .....                            | 23 |
| 图 3-3 | 基于模型一致性的检测评分机制训练流程图 .....                  | 27 |
| 图 4-1 | 动态处理算法系统架构图 .....                          | 33 |
| 图 5-1 | 中值滤波聚合算法系统架构图 .....                        | 37 |
| 图 6-1 | D100 训练流程 .....                            | 45 |
| 图 6-2 | Non-IID+CIFAR10 下攻击前后不同聚合函数的 TACC 对比 ..... | 48 |
| 图 6-3 | IID+CIFAR10 下攻击前后不同聚合函数的 TACC 对比 .....     | 49 |

## 表录

|        |                                         |    |
|--------|-----------------------------------------|----|
| 表 2-1  | Hessian 与 L-BFGS 开销.....                | 19 |
| 表 6-1  | 各模型的鲁棒性组成 .....                         | 45 |
| 表 6-2  | FCNN 架构设置 .....                         | 46 |
| 表 6-3  | CNN 架构设置.....                           | 46 |
| 表 6-4  | 实验环境 .....                              | 47 |
| 表 6-5  | 模型在 CIFAR10 数据集上结合不同聚合函数的测准度 .....      | 50 |
| 表 6-6  | 模型在 CIFAR10 数据集上在不同攻击场景的测准度 .....       | 51 |
| 表 6-7  | 模型在 MNIST 数据集上配合 CNN 模型在不同聚合函数上的测准度 .   | 52 |
| 表 6-8  | 模型在 MNIST 数据集上配合 FCNN 模型在不同聚合函数上的测准度    | 52 |
| 表 6-9  | 模型在 MNIST 数据集上配合 CNN 在不同攻击场景的测准度 .....  | 53 |
| 表 6-10 | 模型在 MNIST 数据集上配合 FCNN 在不同攻击场景的测准度 ..... | 54 |
| 表 6-11 | 中值滤波算法 TACC(%)实验结果.....                 | 57 |

# 1. 绪论

在信息化时代迅猛发展的浪潮中，机器学习技术已深度渗透至社会各领域，在为人类社会带来前所未有的便利与效率的同时，也对信息隐私安全提出了严峻挑战<sup>[1]</sup>。联邦学习（Federated Learning, FL）<sup>[2]</sup>作为一种分布式机器学习范式应运而生，其通过本地数据不出域、仅共享模型参数的方式，有效缓解了传统集中式训练带来的隐私泄露风险。但随着当前开放的网络环境为各类新型网络攻击提供了滋生土壤，传统的联邦学习训练模式面临着前所未有的安全压力。在此背景下，如何有效抵御恶意攻击、保障数据隐私安全，已成为学术界和工业界共同关注的重要课题。论文对非独立同分布 (Not identically and independently distributed, Non-IID) 背景下的联邦学习防御技术进行研究。本章作为论文的第一章，主要介绍论文的研究背景和意义，梳理 Non-IID 背景下的联邦学习防御技术领域的研究现状，并概要介绍本文的主要研究内容及论文组织结构。

## 1.1 研究背景与意义

在大数据与人工智能深度融合的时代背景下，机器学习技术在许多方面出现了突破性进展，比如人工智能与自然语言处理、计算机视觉、个性化推荐等，但这些模型的构建无不需要大量高质量训练数据资源，然而，这种传统的集中式训练模式面临着双重挑战：法律方面，欧洲提出了通用数据保护条例( General Data Protection Regulation, GDPR)<sup>[3]</sup>的以监管数据隐私，国内也于近年颁布了相关的网络安全法和数据安全法，隐私保护意识的提高导致获取高质量数据越来越困难；从训练角度，传统集中式机器学习模式在应对海量分布式数据时，无法有效解决因数据分散导致的“数据孤岛”问题，如果强行集中会导致隐私泄露的风险，并且不同数据库的数据格式不一致也会给训练带来极大的不便。

为了突破这一困境，分布式机器学习就应运而生，但传统分布式方法通常是在实验室理想环境中基于种种理想假设，比如假设数据独立同分布（IID），而实

际应用场景中因为个性化差异,数据分布反而往往呈现显著的 Non-IID<sup>[4]</sup>特性,这种数据分布的异质性严重制约了传统分布式模型在现实场景中的适用性<sup>[5,6]</sup>。

2016年,Google的研究团队创新性地提出了联邦学习(Federated Learning, FL)框架,通过"数据不动模型动"的核心思想,在保护数据隐私的前提下实现了跨节点的协同学习,并且能兼容各个节点之间的差异。该技术已成功应用于 Apple 的 Siri<sup>[7]</sup>、Google 的 Gboard<sup>[8,9]</sup>等商业系统,展现出强大的应用价值<sup>[10]</sup>。该机制的核心逻辑是:各参与方仅需上传本地训练的模型参数而非原始数据,由中央服务器收集各客户端提交的参数以聚合更新全局模型。这种方式既避免了直接的原始数据共享导致泄露的风险,又能够间接从分布式数据合成的参数中学习特征。

尽管联邦学习成功避免了直接共享用户训练数据引发的信息泄露风险的问题,但它仍然面临着多方挑战,包括端到端大量的通信开销、黑客通过逆向参数推理生产原始数据以及拜占庭攻击等问题。这些问题的存在,使得联邦学习如何在隐私安全与模型效能之间的平衡成为当前研究的重点和热点。

拜占庭攻击<sup>[11,12]</sup>是指参与训练的客户端提交了错误信息从而导致全局模型训练失败,它作为分布式系统的经典安全威胁,在联邦学习框架下呈现出新的挑战形式。联邦学习特有的分布式架构使其面临双重拜占庭风险:一方面源于客户端设备在计算或通信过程中可能发生的随机故障导致提交了错误信息;另一方面则来自黑客精心设计的参数篡改行为。这些异常行为会显著干扰全局模型的收敛过程,不仅导致模型性能下降,更可能引发严重的隐私泄露风险,而联邦学习的兼容高 Non-IID 数据的特性,往往使得这些错误或者恶意的数据,难以与独具一格的信息相区分开。Chen 等人<sup>[13]</sup>的研究表明,即便仅存在单个拜占庭节点,也足以导致整个联邦学习系统训练失败。

在防御机制方面,虽然 McMahan 等人提出的联邦平均算法(FedAvg)<sup>[2]</sup>通过梯度平均化确保了模型在正常情况下的收敛性,但其本质上缺乏对抗拜占庭攻击的能力,因为其全盘接受所有信息。为此,研究者们提出了多种基于统计分析和距离度量的防御方案。其中,Blanchard 团队开发的 Krum 算法<sup>[12]</sup>颇具代表性,该算法通过计算节点梯度间的欧氏距离,选择与多数节点最接近的梯度作为聚合基

准。这类方法的核心假设是：节点数据满足独立同分布（IID），且正常节点占据绝对多数，因此可以通过统计特性有效识别异常节点。

然而，现实中的联邦学习场景往往呈现显著的 Non-IID 特性，导致各节点参数分布存在差异。这种天然参数异质性使得基于统计距离的检测方法面临严重误判风险<sup>[14]</sup>：正常节点的独特参数可能被误判为恶意行为，而精心设计的拜占庭攻击数据反而可能伪装成具有合理的数据分布特性。这一矛盾凸显了传统拜占庭防御机制在 Non-IID 环境下的局限性，因此亟需发展新的具有更好鲁棒性的联邦学习训练算法。

## 1.2 研究现状

### 1.2.1 联邦学习研究

传统集中式训练要求参与者将本地数据上传至中央服务器，再由服务器统一整合并训练，不仅耗费人力，还容易泄露数据。这类方法如今也面临着诸多限制：其一是安全法规，随着各国对网络和数据逐渐重视，相应的法律不断被出台以保障个人隐私安全与国家关键数据的安全，比如欧洲的 GDPR<sup>[3]</sup>、我国的《中华人民共和国网络安全法》、《中华人民共和国数据安全法》以及《新一代人工智能伦理规范》，进一步规范了数据采集、存储、传播和使用的情形；其二是个人意愿，随着公民素质的提高，人们越来越重视自己的隐私，不愿意随便把个人数据交付给第三方，对数据采集造成了更大的困难；其三是机构的数据难以共享，当代机构已经意识到数据蕴含的高价值，并不愿意共享一些关键数据或者给与数据于可能造成威胁的公司，敝帚自珍使得数据传播进一步变得艰难。这些情况严重制约了集中式模型训练的发展与应用普及。

由此发展出了传统分布式学习方法，但传统分布式训练方法通常基于许多理想条件，这导致得到的传统分布式训练而得的模型在现实应用场景非常狭窄<sup>[15]</sup>。

在 2017 年，McMahan 等人首次提出了联邦学习（Federated Learning, FL）<sup>[2]</sup>与联邦平均算法（Federated average, FedAvg）的概念，其中中央服务器通过计算用户节点上传梯度的平均值来更新全局模型参数。尽管 FedAvg 方法能确保全局模

型收敛，但无法抵御拜占庭攻击。

在早期的工作中, Richardson 等人<sup>[16]</sup>提出了一种支付机制来平衡和筛选各个子节点, 这种机制结合了激励机制和博弈论, 对提供高质量数据的节点以更多的激励作为回报, 并在理论上证明了能够确保数据质量。

2019 年, Li 等人<sup>[17]</sup>等人提出了一种优化方法(q-Fair Federated Learning, q-FFL)来确保联邦学习模型在各个设备上的表现一致, q-FFL 是 FedAvg 的泛化, 通过引入  $q$  参数使各设备的权重被参数化, 让模型向训练的弱势方移动, 对使得全局模型的损失值较大的参与者给予更大的相对权重, 从而降低在不同用户端上训练的表现方差。q-FFL 构建了一个灵活的框架, 该框架可以根据所需的公平性自动调整目标, 无需手工调整公平性约束。

在论文<sup>[18]</sup>中提出了一种优化算法 (FedFa), 以在联邦学习中实现更进一步的公平性和准确性。FedFa 采用双动量梯度和一个权重选择算法, 双动量梯度用以加速模型的收敛率, 权重选择算法则结合训练精度和训练频率的信息量来计算权重。

### 1.2.2 联邦学习相关的攻击机制研究

前人<sup>[19]</sup>针对分布式学习的拜占庭问题, 提出只要针对性的稍微改变少数客户端节点的许多参数也足以击垮所有现有的防御手段, 并干扰甚至控制训练过程。和以往的假设不同, 此项工作证明了拜占庭节点不需要完全知识背景或者大幅度参数变动就能攻击成功。

论文<sup>[20]</sup>介绍了后门攻击(Backdoor attack)的含义, 形式化地描述了后门攻击情况下, 攻击者对全局模型所造成的影响, 还介绍了后门攻击的防御手段, 如范数约束、差分隐私。

Fang 等人<sup>[21]</sup>提出了模型中毒攻击 (Model poisoning attack, MPA) 的概念, 其通过将损失函数计算转化为函数优化问题, 使恶意客户端每次提交与理想更新方向相反的更新, 从而最大化全局损失。实验结果表明, 攻击不仅对 FedAvg 有效, 还能有效对抗当时声称自己具有鲁棒性的多种聚合方法, 例如 Krum、Trimmed

Mean<sup>[22]</sup>和 Median<sup>[23]</sup>。此外作者还提出了部分知识攻击（partial knowledge attack）和完全知识攻击（full knowledge attack）的说法：部分知识攻击情境下，攻击者可以操控被控制的客户端的一切数据集和训练过程，但是不能直接影响其他良性客户端和中央服务器；完全知识攻击情境下，攻击者知道其它良性客户端的训练情况和提交的数据，但是不能操纵和篡改，这种情况并不常见，往往是安全系统出现了极大漏洞，因此作为一种极限情况加以参考。对于针对性设计出的几种攻击，实验证明在其完全知识背景下攻击的破坏强度更大。

### 1.2.3 联邦学习相关的防御机制研究

联邦学习的鲁棒性<sup>[24]</sup>指的是系统在面对数据无规律和恶意攻击的情况下，依然能够保持稳定运行。以往作者往往假设大部分的节点都是善意的，仅有少数拜占庭节点。Xie 等人<sup>[25]</sup>提出了 Zeno 方法，通过引入一台可信移动设备，利用其更新构建边界，从而有效过滤掉可能来自被攻陷节点的恶意梯度，即便拜占庭节点比例很高，也能保证全局模型收敛。

在论文<sup>[26]</sup>中作者基于拜占庭节点之间会交流以一起欺骗中央节点的假设，讨论了分布式中参与者分布相似的拜占庭容错问题，提出了通过分布式 SGD 和梯度比较的容错机制。

为适配去中心化场景，论文<sup>[27]</sup>设计了节点间的“互评”机制：每位设备对其他设备的更新分别打分，通过累积得分评估可信度，进而加权聚合——但该方案依赖点对点评分与区块链账本，不易推广到有中央服务器的典型联邦学习体系。

另一种思路是干扰与检测，论文<sup>[28]</sup>中作者使用精心制作的高斯噪声来区分各客户端提交的模型参数是否恶意，但其面对海量客户端时运算成本颇高。

论文 FLTrust<sup>[29]</sup>中，作者提出在服务器上额外存储一个根数据集，以各客户端模型在根数据集上的余弦相似度作为声誉分数，然后根据声誉值调整聚合权重。但这个方法一个巨大的难点就在于如何选取该数据集，Non-IID 下规律并不明显，因此很难选取最有代表性的根数据集。如果根数据集选择得不好，那么良性更新的余弦相似度会普遍偏小，也就意味着预测不准。

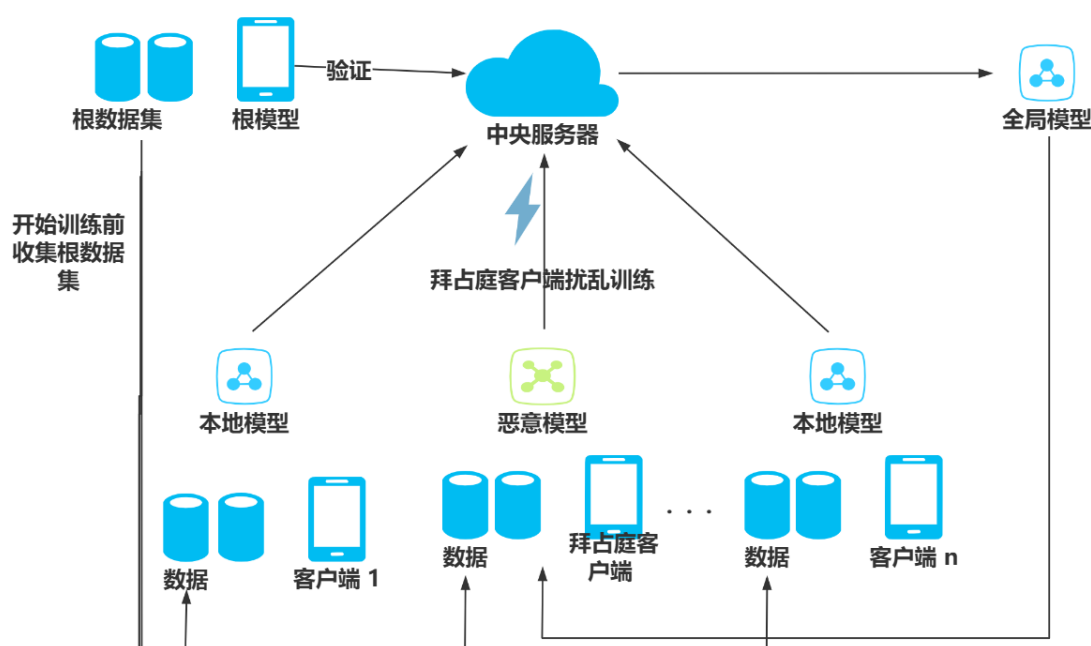


图 1-1 FLTrust 运作图  
Fig 1-1 Diagram of FLTrust process

Zhang 等人<sup>[30]</sup>提出了一种名为 FLDetector 的恶意客户端检测方法。FLDetector 解决了过去鲁棒性较好的训练方法 FLTrust 需要干净根数据集的难题,基于的理论是良性客户端的正常训练更新是有规律的、可预测的,而模型中毒攻击(MPA)的恶意的更新是被设计好的、生硬的、不一致的,所以可以通过更新变化的趋势来判断可疑程度,实验证明这种想法是有效果的。具体方法是,对每个客户端,通过 L-BFGS 去估算 Hessian 矩阵以降低计算成本,再利用估算的 Hessian 矩阵和上一轮梯度来预测本轮梯度更新,通过预测值和真实值欧几里得距离来判断梯度更新的可疑程度。把各客户端的差别给 Gaps 和 K-Means 统计方法进行自监督学习,系统检测到攻击之后根据学习结果,剔除恶意客户端再重启训练,直到最后某次训练中无法检测到攻击之后,配合已有的一些鲁棒聚合算法训练出一个最终模型。

在联邦学习的聚合算法中, FedAvg 算法被广泛用作一种常见的聚合方法。算法的核心思想是将用户上传的模型权重定义为其训练数据样本在所有设备训练数据样本中的占比。然而,研究人员 Zhao 等人<sup>[31]</sup>发现在数据 Non-IID 的情境下, FedAvg 算法在收敛速度和最终精度方面存在一些不足。

Li 等人<sup>[17]</sup>解决了 FedAvg 存在的不公平性问题。他们采用了一种从公平性角度定义的方法，即将较高的权重分配给性能较差的设备。具体而言，他们赋予在本地训练中损失更高的用户设备较高的权重，以使模型的精度分布朝着更均匀的方向发展。这项工作与 Mohri 等人的研究<sup>[32]</sup>有相似之处，但在实现的角度上存在一定的区别。Li 等人通过根据用户设备在本地数据集上的损失分配权重，提出了这种方法。而 Mohri 等人通过为损失更大的用户设备加权组合来进行模型优化。这种方法的目的是通过优化性能较差的设备来提高整体模型精度，但可能导致模型的收敛速度变慢。

Song 等人<sup>[33]</sup>设计了一种基于信任的物联网聚合算法，其考虑用户的数据集质量并且根据各自数据对全局模型训练贡献高低来决定声誉高低，从而影响之后训练的权重。但是这个算法是基于物联网环境的，而且一旦系统改变，就需要重新评估。

Melis 等人<sup>[34]</sup>在探讨协同学习中的梯度泄露问题时指出，攻击者可能通过巧妙调整更新幅度使得其贡献恰好处于防御阈值附近，从而以低廉代价的绕过防御措施，这就是“临界规避”问题。大量绕过防御的恶意数据会对系统造成严重破坏。

#### 1.2.4 当前研究工作总结

综上所述，联邦学习在训练公平与拜占庭攻击<sup>[35]</sup>等方向存在安全隐患。随着攻击手段的逐渐进步，出现了许多新攻击类型，旧有的防御方法已经不能满足安全需要。现有的技术主要有两个大方向：一个是通过区块链共识算法进行协商，但这类算法虽然安全性较好，但是往往算力消耗太大且应用条件较为苛刻，对于联邦学习场景中低算力设备上应用并不友好；另一个是通过检测筛选算法进行处理，旧有的检测算法多数依赖于数据分布之间的共性规律，即假设 Non-IID 的程度不高，各节点参数迭代比较相似，很显然在联邦学习的实际应用场景中，如果出现 Non-IID 程度高的情况，该检测容易误判。如果能设计出一种不依赖于各个节点数据集之间的数据规律的联系来判断善恶的检测方法，那么就能极大缓解因 Non-IID 的程度变化导致的检测失灵。

对于采用“临界规避”策略的攻击者，基于平均的梯度处理思想隐含了各梯度等重要性的假设，当客户端数据存在较大分布差异或梯度方向分散时，平均处理的方法容易使有效信息被稀释与失真并且攻击手段更容易对模型施加强大的影响。相比之下，动态加权<sup>[36,37]</sup>生成的聚合梯度本质上是各可信客户端梯度的一个非均匀凸组合<sup>[38]</sup>，这一组合更好地保留了各个本地更新的几何分布特性，能有效突出重要信息，并且动态加权能很好应对“临界规避”问题，因为攻击者很难预测一个动态的边界。

针对该方面挑战，本文采用联邦学习架构，在中央服务器中引入基于 Hessian 矩阵的模型更新一致性判断方法，其依赖于单个节点前后的训练轨迹的合理性而不依赖于各个节点之间的分布规律进行模型中毒攻击的识别，因此能很大程度规避各数据集 Non-IID 的影响；随后采用动态聚合的思想，结合检测算法进行动态加权处理，以解决“临界规避”问题并提高信息汲取效率；最后，为了提高防御体系的全面性，提出一种可以对部分特定的目标数据中毒攻击起到更好防御效果的聚合算法，并和第一种算法形成互补关系。通过构建三种算法形成的全面防御体系，从而达到维护信息安全的目的。

### 1.3 研究内容

本论文主要研究 Non-IID 下联邦学习的针对拜占庭攻击的具有鲁棒性的防御方法，以抵御攻击的能力（鲁棒性）和对 Non-IID 程度的适应为主要研究对象。针对之前研究中已有模型的不足，论文研究一种同时满足以上两点的新型具有较好鲁棒性的防御技术，保证其在 Non-IID 的不同程度下都能保持较好的防御能力。具体研究内容如下：

（1）论文研究了基于 Hessian 矩阵实现的通过模型更新一致性的节点评分机制，将客户端的本地模型训练过程模拟成基于 Hessian 矩阵的迭代过程。模型更新一致性的思路是对于攻击尤其是模型中毒攻击（MPA），攻击者的刻意性和目的性是很强的，攻击者要构造出符合他想法的梯度更新交给中央服务器以破坏全局模型，那么设计出的梯度之间很难像正常训练出的梯度之间有一定的趋势关联。

基于 Hessian 矩阵的预测机制就是依据之前轮次该客户端本地模型的梯度变化趋势得出一个 Hessian 矩阵，来近似本轮梯度更新的变化大小和方向，结合上一轮实际梯度更新进行预测，以此和实际梯度更新进行对比，计算其欧几里得距离并结合其近期历史情况综合得出该客户端本轮评分。考虑到直接计算 Hessian 矩阵的复杂性，为了降低每轮 Hessian 矩阵计算的算力消耗，引入了 L-BFGS 算法去以较低代价近似 Hessian 矩阵。如此就得到了一个几乎不受 Non-IID 影响且算力代价低廉的检测评分机制。

(2) 论文研究了基于可疑分数的动态处理梯度算法，该算法的思想主要归纳于以下三点原则：屏蔽恶意客户端：对确认为恶意的客户端，暂时屏蔽其更新；谨慎采纳低可信客户端：对可信度较低的客户端，少量采纳其信息，降低潜在风险；充分采纳高可信客户端：对可信度较高的客户端，赋予更高的权重，充分采纳其信息。其流程是首先引入恶意门槛，基于恶意门槛分割善恶客户端，利用良性客户端与恶意门槛的差值作为权重，构造本轮填充梯度以代替恶意客户端本轮参与聚合的梯度更新，然后提交给后续选定的聚合算法进行处理。

(3) 论文在前文模型的基础上，提出一种中值滤波聚合算法，在聚合时考虑周围参数一起参与，并在求中位数时使用归一化和反归一化消除度量差异造成的影响，从而进一步提高攻击的难度和成本。实验结果验证了本文提出的该聚合算法在一些情景下具有不错的能力。

(4) 论文采用机器学习领域常用的数据集 MNIST 和 CIFAR10 进行实验，通过引入不同攻击类型和聚合算法结合本论文机制在两种 Non-IID 情境下进行大量实验，验证了论文提出的基于模型更新一致性的节点评分、基于可疑分数的动态处理梯度算法、中值滤波聚合算法在联邦学习的针对拜占庭攻击防御方面的可靠性。

各算法防御目标与相互关系如图 1-2 所示。

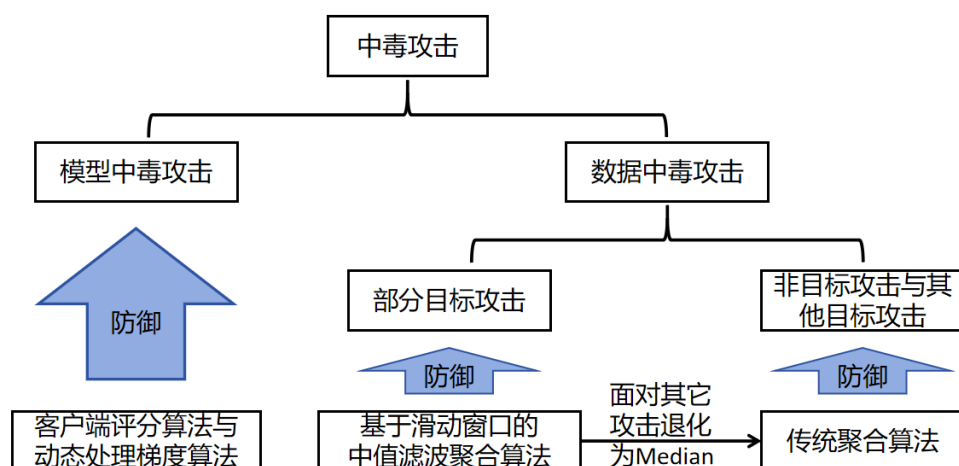


图 1-2 防御系统架构  
Fig 1-2 The defense system architecture

## 1.4 组织结构

根据以上研究内容，论文分为六个章节，整体的组织结构如下。

第一章，绪论。本章首先阐述了课题的研究背景与意义，包括联邦学习发展现状以及 Non-IID 下防御技术的意义。然后分别从联邦学习的发展、攻击技术的发展与防御技术的发展综合讨论了联邦学习具有鲁棒性的防御技术的研究现状。最后对论文的研究内容和组织结构进行了介绍。

第二章，相关技术介绍。本章主要对论文中所涉及到的技术进行了详细的介绍。介绍了包括联邦学习的攻击与防御技术、全连接神经网络、卷积神经网络、Hessian 矩阵与 L-BFGS 算法，最后对论文中所使用到的数据集做了详细的描述。

第三章，提出一种基于模型更新一致性的节点评分机制，从而在较低算力代价的基础上避免了数据高 Non-IID 对检测精度的影响。本章介绍了其数学原理，并对其流程进行梳理。

第四章，提出一种基于恶意评分的动态处理梯度机制，从而最大化兼顾训练信息提取的效率和安全性。本章介绍了其运作流程并进行了原理分析。

第五章，提出一种基于滑动窗口的中值滤波聚合算法，针对一些目标攻击场景，规避了 Median 聚合算法容易被大量恶意客户端操纵从而破坏鲁棒性的风险。

本章介绍其理论场景与运作原理，随后介绍算法细节。

第六章，对实验流程和结果进行说明。本章首先对实验环境和设置进行了全面阐述，随后对四五六章中提出的三种算法在 MNIST 和 CIFAR10 数据集上针对两种 Non-IID 程度、多种聚合和攻击算法进行了广泛的实验，随后对实验结果进行对比和深度分析，从而证明了本文提出的算法在特定场景下的优越性。

第七章，结论与展望。本章对论文的工作内容进行了总结，并对 Non-IID 下联邦学习的针对拜占庭攻击防御机制的发展进行了展望。

## 2. 相关技术介绍

随着机器学习模型在生活中的大量应用，人们越来越意识到高价值数据集与其相关的隐私安全的重要性，其热度与潜藏的数据价值吸引了许多不法分子的注意，针对模型的攻击手段也越来越复杂。相对的，防御手段也是不断进步、日新月异。本章主要介绍了联邦学习下的与拜占庭攻击相关的重要攻击与防御方法的概念以及论文中所使用到的相关技术方法，为后续章节的研究工作奠定理论基础。

### 2.1 联邦学习

#### 2.1.1 联邦学习概念

如图 2-1 所示，联邦学习(FL)是一种不需要共享私有数据集的协作机器学习架构。一般该方法中存在两种角色，第一个是中央服务器（Central Server），该角色通常由某些服务提供商提供的高性能计算机所承担；另一个是客户端（Client），它通常由资源有限的边缘计算设备组成。各客户端在本地训练后，将模型参数（或梯度）定期上传至中央服务器，服务器对所有有效更新进行聚合并生成新的全局模型，再分发至各客户端。此过程既避免了集中式学习对全量数据的收集、存储和处理，也无需在数据中心投入海量算力。以 Google 提出的 FedAvg 为例，其通过对各客户端上传的本地模型参数按本地样本数加权平均，形成全局模型更新，使得全局模型能够在不接触原始数据的情况下，间接学习到所有私有数据的特征。

传统的联邦学习框架其步骤可以分为以下四步：

- a) 本地模型训练：各客户端基于各自的本地数据集进行模型训练，生成个性化本地模型。
- b) 参数传输：各客户端将训练所得的模型参数或梯度更新信息传输至中央服务器。
- c) 全局聚合：中央服务器通过聚合算法整合所有客户端提交的参数更新，形成当前训练轮次的全局模型。

d) 模型同步: 聚合后的全局模型由服务器分发给所有参与客户端, 作为下一轮次训练的初始模型, 该迭代过程持续进行直至中央服务器发出终止指令。

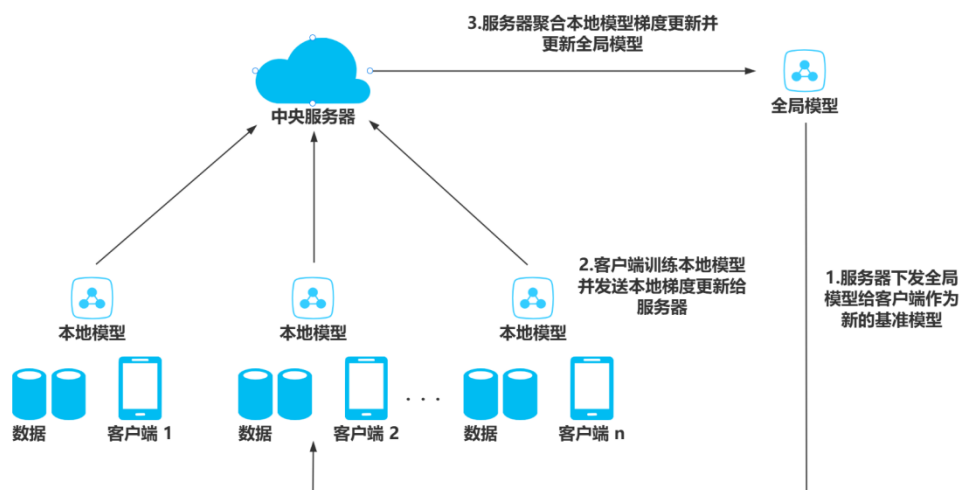


图 2-1 联邦学习训练流程示意图  
Fig 2-1 Diagram of Federated Learning training process

### 2.1.2 联邦学习防御介绍

为了抵御不断涌现的形形色色的拜占庭攻击手段, 前人提出了很多具有一定鲁棒性的方法, 比如 Trimmed Mean、Krum、Median 等聚合算法, 或者利用干净根数据集来验证更新的 FLTrust<sup>[29]</sup>和 Zeno<sup>[25]</sup>等。方法大致可分为两类。一种是通过异常检测方法识别和删除潜在的中毒更新, 这些检测或者删除机制往往利用统计分布或者针对攻击者使用的策略来运行, 缺点是检测方法往往有局限性, 大量依赖统计分布, 在高 Non-IID 情况下不够准确<sup>[39]</sup>; 另一种是将基于区块链<sup>[40]</sup>的共识算法来排除不合理因素, 比如将拜占庭容错 (Practical Byzantine Fault Tolerance) 分布式学习<sup>[41-44]</sup>中开发的现有技术迁移到 FL 环境中, 他们通常假设服务器可以获得最大数量的异常客户端, 然后利用区块链技术和共识算法对每一次更新进行协商, 只有完全达成共识才会采纳更新, 这种方法虽然鲁棒性不错, 但是有一些的缺点:

1. 算法过程比较复杂、算力需求和耗时很大<sup>[45,46]</sup>, 网络不能太大且不易拓展, 对于联邦学习下的低算力设备不友好<sup>[41]</sup>;

2.一些失效的节点副本容易被黑客获取，导致泄露信息。

### 2.1.3 联邦学习攻击介绍

联邦学习虽然通过分布式架构有效解决了数据集共享引发的隐私泄露问题，但其安全性仍面临着各类攻击的严峻挑战。由于中央服务器无法直接验证原始数据的真实性，导致联邦学习的运行安全依赖于客户端的可靠性，这使得原始联邦学习系统面对攻击尤为脆弱。

拜占庭攻击其名称来源于拜占庭将军问题<sup>[11]</sup>，其流程如图 2-2 所示，单来说就是在分布式模型训练中出现被黑客操纵或者利用的恶意节点（客户端），该恶意节点就被称为拜占庭节点，利用该拜占庭节点进行对全局模型的破坏就叫拜占庭攻击。在联邦学习中，拜占庭攻击往往通过中毒攻击的形式来实现。

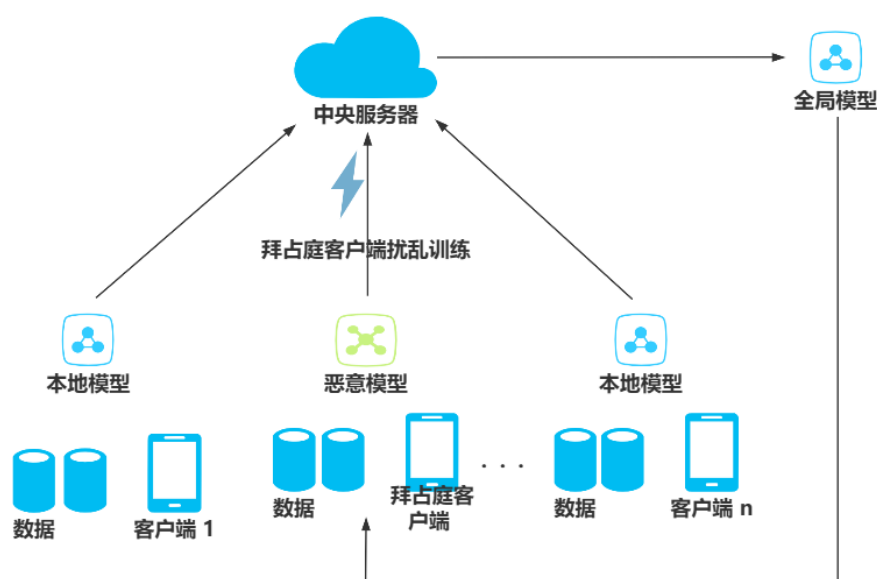


图 2-2 拜占庭攻击示意图  
Fig 2-2 Byzantine Attack Diagram

联邦学习的中毒攻击，顾名思义，是去毒害训练中的全局模型从而使模型最终达到黑客想要的效果。

联邦学习的中毒攻击从目的的角度可以分成目标中毒攻击<sup>[47]</sup>和非目标中毒攻击<sup>[48,49]</sup>，目标中毒攻击，是怀着特定目的去尝试操纵全局模型，比如在模型分类任

务中，故意误导全局模型把狗的图片分类成猫，最典型的类型就是后门攻击<sup>[50-52]</sup>及其衍生攻击如分布式后门攻击（DBA）<sup>[53]</sup>；而非目标中毒攻击，则是攻击者并不故意引导全局模型做出某些特定行为，而是广泛的破坏全局模型的训练，造成模型的不可用。

联邦学习的中毒攻击，从训练阶段的角度来看，可以分为数据中毒攻击<sup>[54]</sup>和模型中毒攻击<sup>[21,55]</sup>，如图 2-3 所示。数据中毒攻击就是在联邦学习训练的前置阶段--数据采集的时候，通过篡改或者混入一些具有破坏性的或者不符合训练目的和常理的数据，从而去误导本地模型并让本地模型提交破坏性参数，达到攻击者的目的。而模型中毒攻击由 Fang 等人提出，旨在通过直接构建或控制本地模型训练来提交误导性信息，从而破坏全局模型。典型的例子就是，Fang 等人<sup>[21]</sup>提出的攻击方法，包括修剪均值（Trim）攻击、反向梯度攻击（Mean Attack）、高斯攻击（Gaussian Attack）和 Krum 攻击。

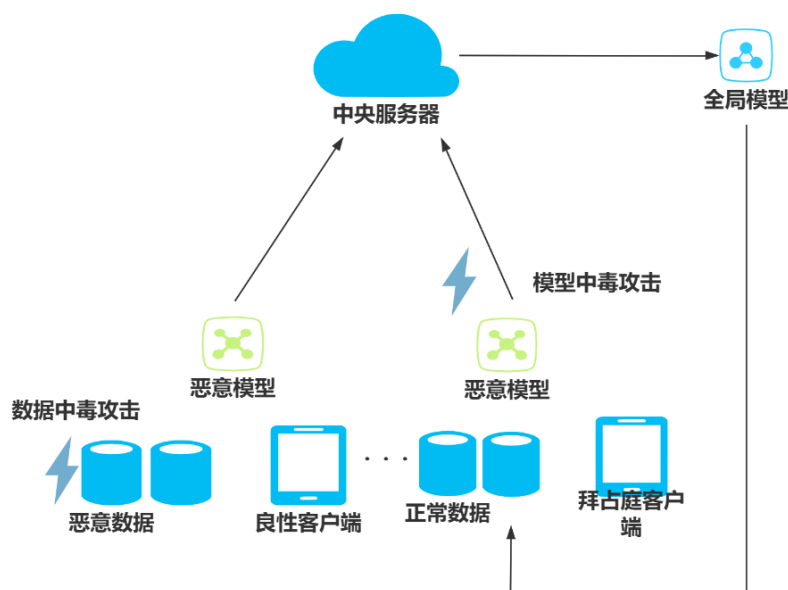


图 2-3 数据和模型中毒攻击示意图  
Fig 2-3 Diagram of data and model poisoning attacks

## 2.2 Hessian 矩阵

Hessian 矩阵 (Hessian Matrix) 最早是 19 世纪的德国数学家 Ludwig Otto Hesse 提出并命名, Hessian 矩阵从性质上来说是多维函数的二阶偏导数矩阵, 用于描述多元函数的局部曲率, 由于是二阶导数, 它提供了比一阶导数 (梯度) 更多的函数趋向信息。它被用来结合正定负定性判定极值且常常被应用在牛顿法之类的近似任务中。

对于一个二阶可微的多维函数  $f(\mathbf{x})$ , 其中  $\mathbf{x}=(x_1, x_2, \dots, x_n)^T$ , 该函数的 Hessian 矩阵  $H$  就是一个  $n \times n$  的对称矩阵。

$$H(f) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \quad (2-1)$$

即:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j} \quad (2-2)$$

Hessian 矩阵的特征值可以判断函数的局部几何性质:

正定 (Positive Definite): 所有特征值  $> 0 \rightarrow$  局部极小值。

负定 (Negative Definite): 所有特征值  $< 0 \rightarrow$  局部极大值。

不定 (Indefinite): 有正有负  $\rightarrow$  鞍点 (Saddle Point)。

半定 (Semi-Definite): 部分特征值  $= 0 \rightarrow$  无法直接判断。

### 2.2.1 Hessian 矩阵在梯度近似方面的理论

设全局损失函数为  $\mathcal{L}(\theta)$ , 客户端的局部损失为  $\mathcal{L}_i(\theta)$ 。全局梯度与本地梯度的偏差可近似为:  $\nabla \mathcal{L}(\theta) - \nabla \mathcal{L}_i(\theta) \approx H_i(\theta - \theta^*)$ , 其中  $H_i$  是对应客户端  $i$  的本地 Hessian 矩阵,  $\theta^*$  为全局最优参数, 可预测全局梯度的变化趋势。

柯西中值定理指出, 对于连续可微函数, 存在某点  $\xi$  使得:

$$f(b) - f(a) = f'(\xi)(b - a) \quad (2-3)$$

在多元情况下,积分形式的 Hessian 矩阵  $H_i^t$  可以看作是梯度的“平均变化率”,类似于—维中的  $f'(\xi)$ 。根据柯西中值定理(泰勒展开一阶近似)类比,对客户端  $i$  的损失函数  $\mathcal{L}_i(w)$  在参数  $w_{t-1}$  进行二阶泰勒展开,得:

$$\mathcal{L}_i(w_t) \approx \mathcal{L}_i(w_{t-1}) + \nabla \mathcal{L}_i(w_{t-1})^\top (w_t - w_{t-1}) + \frac{1}{2} (w_t - w_{t-1})^\top \nabla^2 \mathcal{L}_i(w_{t-1}) (w_t - w_{t-1}) \quad (2-4)$$

其中:  $\nabla \mathcal{L}_i(w_{t-1})^\top$  是梯度(一阶导数),而  $\nabla^2 \mathcal{L}_i(w_{t-1})$  是 Hessian 矩阵(二阶导数)。对泰勒展开两边对  $w_t$  取梯度,得到:

$$\nabla \mathcal{L}_i(w_t) \approx \nabla \mathcal{L}_i(w_{t-1}) + \nabla^2 \mathcal{L}_i(w_{t-1}) (w_t - w_{t-1}) \quad (2-5)$$

该结果适用于  $w_{t-1}$  与  $w_t$  十分接近的情况,如果当  $w_t - w_{t-1}$  的差值比较大时,直接使用上述 Hessian 矩阵会带来巨大的误差。因此改用积分形式的 Hessian 矩阵,即在参数更新路径上对 Hessian 取平均:

$$H_i^t = \int_0^1 \nabla^2 \mathcal{L}_i(w_{t-1} + x(w_t - w_{t-1})) dx \quad (2-6)$$

代入泰勒展开后的公式得:

$$\nabla \mathcal{L}_i(w_t) = \nabla \mathcal{L}_i(w_{t-1}) + H_i^t (w_t - w_{t-1}) \quad (2-7)$$

而  $\nabla \mathcal{L}_i(w_t) = g_i^t$ ,  $\nabla \mathcal{L}_i(w_{t-1}) = g_i^{t-1}$ , 所以得:

$$g_i^t = g_i^{t-1} + H_i^t (w_t - w_{t-1}) \quad (2-8)$$

泰勒展开提供了梯度变化的局部近似,但仅适用于小步长,而积分形式的 Hessian 通过路径平均,更准确地描述了大步长下的梯度变化。公式 2-8 的最终目的是通过 Hessian 矩阵预测当前轮次的梯度  $g_i^t$ ,从而检测恶意客户端(预测与实际梯度偏差过大)。

### 2.2.2 L-BFGS 算法在计算 Hessian 矩阵方面的理论

上文已经证明了 Hessian 矩阵在预测梯度方面的可行性,但是由于其涉及到许多高阶复杂计算,比如糅合了大量二阶导、积分、大型矩阵运算等,导致直接利

用 Hessian 矩阵对大量数据进行计算造成的算力负担非常大。L-BFGS (Limited-memory Broyden - Fletcher - Goldfarb - Shanno) 是一种拟牛顿法, 用于高效近似 Hessian 矩阵或其逆矩阵, 从而避免直接计算和存储高维二阶导数矩阵。

对于普通的 Hessian 矩阵计算, 如果数据量为  $d$ , 那么矩阵存储复杂度为  $O(d^2)$ , 对于大型数据集来说, 这个消耗是难以接受的。拟牛顿法的主要思想是通过一阶梯度的变化信息来近似 Hessian 矩阵  $H$  或者其逆矩阵, 而不是计算出所有的二阶导, 由此引出 BFGS 算法。

依据上文公式可得:

$$\nabla \mathcal{L}_i(w_t) - \nabla \mathcal{L}_i(w_{t-1}) + H_t(w_t - w_{t-1}) \quad (2-9)$$

若设  $H_t$  的逆矩阵  $H_t^{-1}$  为  $B_t$ , 设  $s_t = w_{t+1} - w_t$  是参数更新向量,  $y_t = \nabla \mathcal{L}_{t+1} - \nabla \mathcal{L}_t$  是梯度变化向量。

标准的 BFGS 算法通过以下公式进行自我迭代以计算逼近  $H_t^{-1}$  的  $B_t$ 。

$$B_{t+1} = \left( I - \frac{s_t y_t^\top}{y_t^\top s_t} \right) B_t \left( I - \frac{y_t s_t^\top}{y_t^\top s_t} \right) + \frac{s_t s_t^\top}{y_t^\top s_t} \quad (2-10)$$

虽然 BFGS 免除了大量的高阶导数计算, 仍然需要存储稠密矩阵  $B_t$ , 它和  $H_t$  一样都是需要  $O(d^2)$  存储消耗, 面对大量高维参数会导致消耗巨大, 所以需要办法来降低存储复杂度, 由此引出了 L-BFGS 算法, 其中的 L 就是 Limited-memory (有限内存) 的意思。

L-BFGS 的核心思想是保存最近  $m$  组对  $\{s_t, y_t\}$ , 并动态根据需要进行计算  $B_t$  从而得到 Hessian 的近似值, 降低存储消耗到  $O(md)$ , 其过程如下:

(1) 递归计算 Hessian

$$B_t \nabla \mathcal{L}_t = \text{LBFGS-Recursion}(\{s_{t-i}, y_{t-i}\}, \nabla \mathcal{L}_t) \quad (2-11)$$

其每次递归具体过程包括以下两步:

1) 前向循环: 按时间顺序 (从最旧的历史向量对开始), 应用向量对的秩-1 更新, 相当于逐步修正初始矩阵  $B_t^0$ 。

2) 反向循环: 按时间逆序 (从  $t-m$  到  $t-1$ ) 应用向量对的秩-1 更新。这么做

的原因是拟牛顿法的公式具有时间依赖性，可以让比较新的数据更多影响结果，从而更准确近似结果。

名词解释：其中，秩-1 更新是指通过外积 $s_t y_t^T$ 或 $y_t s_t^T$ 对矩阵进行低秩修正，低秩修正（Low-Rank Update）是优化算法中通过低秩矩阵（如秩-1 或秩-2 矩阵）调整当前矩阵的近似值，而非直接重新计算完整矩阵的手段。

算法解释：每次递归更新本质上是有限步的秩-2 修正，通过 Sherman - Morrison 公式实现。

$$B_{t+1} = B_t - \frac{B_t y_t s_t^T + s_t y_t^T B_t}{y_t^T s_t} + \left(1 + \frac{y_t^T B_t y_t}{y_t^T s_t}\right) \frac{s_t s_t^T}{y_t^T s_t} \quad (2-12)$$

表 2-1 Hessian 与 L-BFGS 开销  
Table 2-1 Expense of Hessian and L-BFGS

| 特性    | 直接计算 Hessian    | L-BFGS 近似             |
|-------|-----------------|-----------------------|
| 存储复杂度 | $O(d^2)$ （不可行）  | $O(md)$ （ $m \ll d$ ） |
| 计算复杂度 | $O(d^3)$ （求逆昂贵） | $O(md)$ （向量内积为主）      |
| 通信开销  | 传输 $O(d^2)$ 矩阵  | 仅传输 $O(md)$ 向量对       |
| 适用场景  | 小规模参数           | 联邦学习等分布式优化            |

两种算法的开销如表 2-2，可以看到 L-BFGS 在时间和空间上都有很大优势。

## 2.3 数据集介绍

论文采用的数据集为 MNIST<sup>[56]</sup>和 CIFAR10<sup>[57]</sup>。

MNIST（Modified National Institute of Standards and Technology）数据集是机器学习领域最经典的入门数据集之一，由美国国家标准与技术研究院（NIST）收集并调整格式，Yann LeCun 等人将其作为基准数据集推广。

MNIST 一共包括 0~9 共 10 个类别的手写数字的灰度图片，灰度范围为 0~255（其中 0 代表黑色，255 代表白色），每张图片大小为 28\*28 像素。其训练集包括 60000 张图片，测试集包括 10000 张图片，主要场景是通过数字识别分类任务来检测一些模型的基础性能。其有一些进阶变体如 Fashion-MNIST<sup>[58]</sup>：替代 MNIST 的 10 类服饰图片数据集，难度略高；EMNIST<sup>[59]</sup>：扩展至字母和更大规模的手写

字符。

MNIST 相对来说可以初步判断一个模型的基础性能，但是很多现代模型能轻易在 MNIST 取得很好的成绩，并且灰度图片的应用场景更为有限。所以本文还引入了非常常用且应用更广泛的 CIFAR10 数据集。

CIFAR10 计算机视觉领域另一个经典的小规模彩色图像分类数据集，常被用作机器学习进阶训练的基准，由加拿大高级研究所 (CIFAR) 整理，Alex Krizhevsky 等人<sup>[60]</sup>推广。

CIFAR10 包括 10 个类别的 32\*32 像素的 RGB 三通道彩色图像，每通道范围为 0~255，每个类别有 6000 张，总共 60000 张。其中训练集一共 50000 张，每个类别 5000 张，测试集一共 10000 张，每个类别 1000 张。这 10 个类别都是生活中常见的物体，分别是飞机 (airplane)、汽车 (automobile)、鸟 (bird)、猫 (cat)、鹿 (deer)、狗 (dog)、青蛙 (frog)、马 (horse)、船 (ship)、卡车 (truck)。其在数据集中用 0~9 分别代表上述的 10 个类别。

## 2.4 本章小结

本章主要阐述了 Non-IID 的联邦学习领域下的一些相关理论以及一些实验中涉及到的技术。首先，介绍了联邦学习的常用框架与运作模式，介绍了前人的一些攻击和防御的思路。其次，介绍了实验中使用到的卷积神经网络和全连接神经网络。最后，介绍了实验中涉及到的 MNIST 和 CIFAR10 数据集，对它们的特性和规模进行了一些阐述，为后续的实验提供了数据来源。

### 3. 基于模型更新一致性的客户端评分算法

联邦学习的恶意客户端检测问题是希望联邦学习模型能在 Non-IID 的数据集下实现训练的同时将潜藏的可能发动攻击的恶意客户端以评分的形式展示出来。本章对设计的一种能够不受 Non-IID 直接影响的基于模型更新一致性的客户端评分算法进行介绍，然后介绍其运作过程。

#### 3.1 算法概述

本章主要研究的是在联邦学习中为了抵御拜占庭攻击中的模型中毒攻击如何去辨别哪些信息是恶意的。文章提出一个区别于传统基于数据集分布检测方法的思路，从而使模型能在数据集 Non-IID 下依旧能保持很好的检测能力，且相对于区块链共识机制拥有更简单的流程和更低的算力消耗。

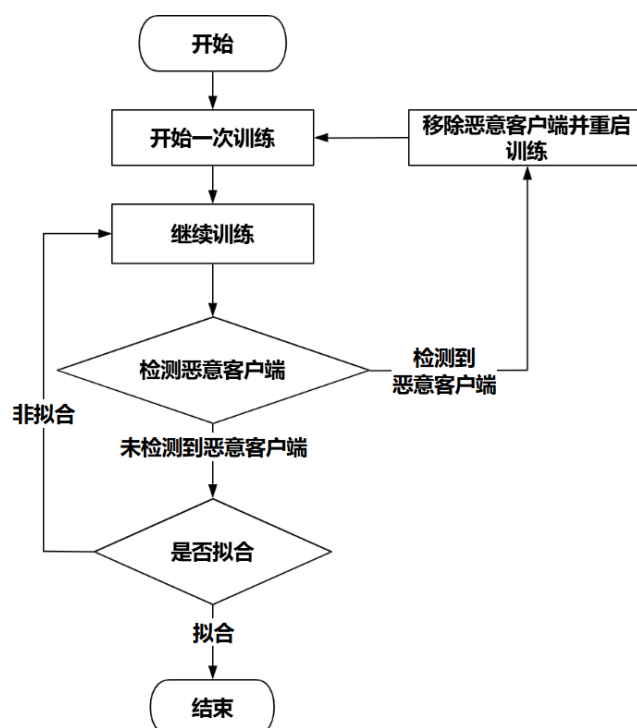


图 3-1 FLDetector 流程图

Fig 3-1 FLDetector flowchart

Zhang 等人<sup>[30]</sup>提出的 FLDetector 的核心理论基于良性客户端更新具有规律性，而模型投毒攻击（MPA）的恶意更新具有突变性。如图 3-1 所示，该方法通过 L-BFGS 近似 Hessian 矩阵预测客户端更新方向，通过预测梯度与实际梯度的差异度评估可信度，结合自监督学习检测并剔除恶意客户端，最终结合鲁棒聚合算法完成训练。实验验证了该思路的有效性。

但该方法存在三方面局限：首先，其重启训练机制在攻击者采用轮换策略时效率显著降低——少量恶意客户端交替攻击会导致模型反复重启，而联邦学习固有的大规模客户端特性会放大效率损失。其次，真实场景中客户端网络的不稳定性（频繁断连重连）会与攻击检测机制形成冲突，极端情况下可能无法完成训练。最后，其刚性剔除策略对阈值附近节点的误判缺乏容忍度，本文建议对此类节点采用渐进式参与机制，通过持续监控和动态调整实现安全与效能的平衡。

此前也有一些作者研究了 Non-IID 下的训练机制，比如 Liu 等人<sup>[61]</sup>提出了 FairFed 模型，其旨在 Non-IID 下根据客户端训练贡献计算 Shapley 值来控制聚合，从而提高联邦学习训练的公平和效率，不过该模型只是在 Non-IID 场景下做训练优化，而非考虑到对抗场景，不具有防御功能。

本章使用的检测方法基于 FLDetector 中的部分原理并重新设计了训练流程，以弥补其部分缺点，其系统架构如图 3-2 所示。架构中包括多个参与者和一个可信的中央服务器，参与者中可能存在一些客户端被黑客所控制，流程一般包含以下 6 个核心步骤：

- a) 本地模型训练：各客户端基于各自的本地数据集进行模型训练，生成个性化本地模型。
- b) 参数传输：各客户端将训练所得的模型参数或梯度更新信息传输至中央服务器。
- c) 评分算法：基于模型更新一致性进行预测并计算与实际值 N 轮的平均欧几里得距离作为可疑评分。
- d) 梯度处理：对达到分数门槛的良性梯度计算均值代替判定为恶意的梯度。
- e) 全局聚合：中央服务器通过聚合算法整合所有梯度更新，形成当前训练轮

次的全局模型。

f) 模型同步：聚合后的全局模型由服务器分发给所有参与客户端，作为下一轮次训练的初始模型，该迭代过程持续进行直至中央服务器发出终止指令。

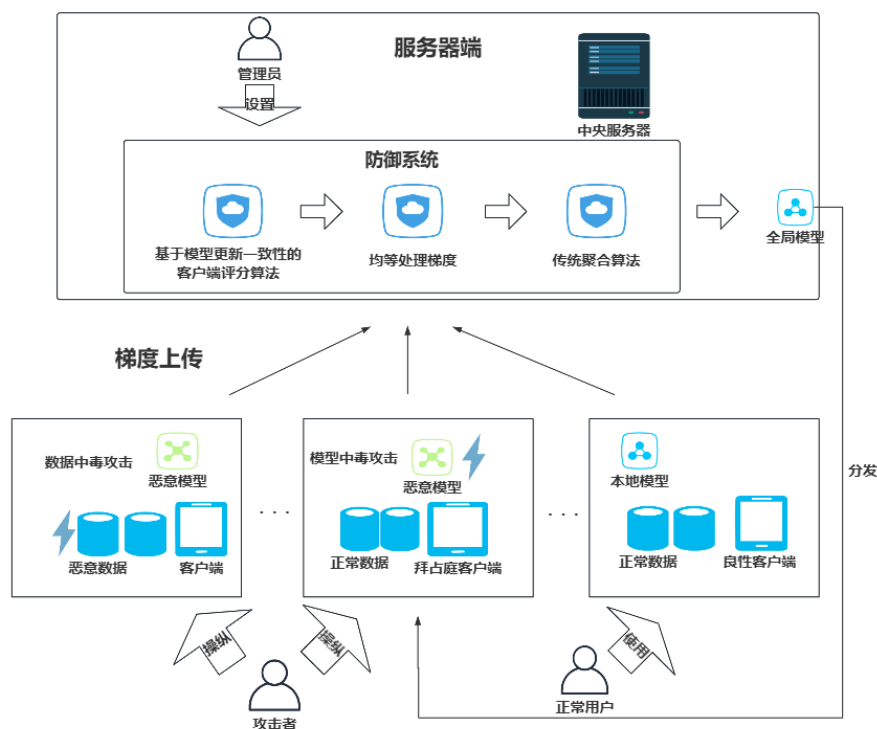


图 3-2 评分算法系统架构图  
Fig 3-2 The system architecture of scoring algorithm.

## 3.2 基于模型更新一致性的客户端评分算法

### 3.2.1 实验模型框架

我们的基本架构和经典 FL 流程一致。假设我们有  $n$  (默认 100) 个客户端且每个客户端有一个本地训练数据集  $D_i, i=1,2,\dots,n$ ，我们用  $D = \bigcup_{i=1}^n D_i$  来表示所有数据集的总集合。训练的目标是得到一个全局模型  $w^*$  使得  $w^* = \arg \min_w \sum_{i=1}^n f(D_i, w)$ ，其中  $f(D_i, w)$  是每个客户端第  $i$  次训练的损失。

初始化的全局模型为  $w_0$ ，在每个轮次  $t$  开始前，会把当前的全局模型  $w_t$  发送给客户端作为训练初始模型，在本地训练并计算  $f(D_i, w_t)$  的梯度  $g_i^t$  作为更新提交给全局模型。

$$g_i^t = \nabla f(D_i, w_t) \quad (3-1)$$

在有些其他的梯度下降方法中, 比如随机梯度下降(SGD<sup>[62,63]</sup>), 会存在客户端训练多轮再汇总成一个梯度更新发送回服务器, 这样会提高通信效率, 但是考虑到其并不会影响我们指标的衡量并且为了方便进行对比, 我们还是和 FLDetector 的实验一样采用单轮梯度下降。

全局模型接收到所有客户端发送的更新后, 服务器基于设定的学习率 $\alpha$ 和安全更新规则 $w_{t+1} = w_t - \alpha g_t$ 计算得出新的全局模型的参数, 以此更新全局模型。

### 3.2.2 模型更新一致性在实验中的应用

---

#### 算法 1: 通过 L-BFGS 计算 Hessian 矩阵

---

**输入:** 全局模型差别  $\Delta W_t = \{\Delta w_{t-N}, \Delta w_{t-N+1}, \dots, \Delta w_{t-1}\}$ , 全局模型更新差别  $\Delta G_t = \{\Delta g_{t-N}, \Delta g_{t-N+1}, \dots, \Delta g_{t-1}\}$ , 向量  $v = w_t - w_{t-1}$ , 历史窗口大小  $N$ ,  $T$  代表矩阵转置,  $t$  表示第  $t$  轮。

**输出:** 近似 Hessian 矩阵  $\hat{H}_t$

- 1 计算  $\Delta W_t^T \Delta W_t$  and  $\Delta W_t^T \Delta G_t$
  - 2 获取  $\Delta W_t^T \Delta G_t$  的对角矩阵  $D_t$  和下三角矩阵  $L_t$
  - 3 计算  $\sigma = \Delta g_{t-1}^T \Delta w_{t-1} / (\Delta w_{t-1}^T \Delta w_{t-1})$
  - 4 获取  $\sigma \Delta W_t^T \Delta W_t + L_t D_t L_t^T$  的 Cholesky 因子  $J_t J_t^T$
  - 5 计算  $q = \begin{bmatrix} -D_t^{1/2} & D_t^{-1/2} L_t^T \\ 0 & J_t^T \end{bmatrix}^{-1} \begin{bmatrix} D_t^{1/2} & 0 \\ D_t^{-1/2} L_t^T & J_t \end{bmatrix}^{-1} \begin{bmatrix} \Delta G_t^T v \\ \sigma \Delta W_t^T v \end{bmatrix}$
  - 6  $\hat{H}_t = (\sigma v - [\Delta G_t \quad \sigma \Delta W_t] q) / v$
- 

我们遵循 FLDetector 中使用 Hessian 矩阵的预测机制。根据柯西中值定理, 我们可以得到以下公式:

$$g_i^t = g_i^{t-1} + H_t(w_t - w_{t-1}) \quad (3-2)$$

其中,  $g_i^t$  是客户端  $i$  在第  $t$  轮迭代中的梯度并且  $w_t$  是  $t$  轮的全局模型,  $H_t^i = \int_0^1 H_i(w_{t-1} + x(w_t - w_{t-1})) dx$  是 Hessian 矩阵  $H_t$  的推导公式, 表示客户端  $i$  的梯度  $g_i^t$  和  $g_i^{t-1}$  之间的一致性。

使用 L-BFGS 算法来优化计算 Hessian 矩阵，可以得到两个推导公式。实际的 Hessian 推导公式是公式 3-2，而结合章节二中的 L-BFGS 的推导公式与算法 1 可以得到如下公式：

$$\hat{g}_i^t = g_i^{t-1} + \hat{H}_t(w_t - w_{t-1}) \quad (3-3)$$

对于 L-BFGS 近似的 Hessian 矩阵，新轮次的梯度与预测梯度之间通常具有较高的相似性。然而，对于恶意客户端，无论近似是否准确，由于其梯度的非自然性，恶意梯度通常与预测值存在显著差异。利用这一点可以进行区分。

### 3.2.3 基于模型一致性的检测机制

基于上面的更新一致性原理，可以计算出每个客户端的预测值  $\hat{g}_i^t$ 。用客户端  $i$  的本地更新预测值  $\hat{g}_i^t$  和实际本地更新  $g_i^t$  求欧几里得距离得到该客户端  $i$  在这一个轮次  $t$  的偏差  $\|\hat{g}_i^t - g_i^t\|$ ， $n$  个客户端的偏差组成偏差向量  $d^t$ ，对这个偏差向量归一化  $\hat{d}^t$  来规范化。文章把归一化向量  $\hat{d}^t$  中的分数称为原始可疑分数，最后结合历史  $N$  轮情况，求最新  $N$  轮的偏差向量的均值就是  $t$  轮第  $i$  个客户端的（最终）可疑分数  $s_i^t$ 。

定义欧几里得距离向量  $d^t$ ：

$$d^t = [\|\hat{g}_1^t - g_1^t\|_2, \|\hat{g}_2^t - g_2^t\|_2, \dots, \|\hat{g}_n^t - g_n^t\|_2] \quad (3-4)$$

归一化向量  $\hat{d}^t$ ：

$$\hat{d}^t = \frac{d^t}{\|d^t\|_1} \quad (3-5)$$

计算可疑分数  $s_i^t$ ：

$$s_i^t = \frac{1}{N} \sum_{r=0}^{N-1} \hat{d}_i^{t-r} \quad (3-6)$$

目前得到的可疑分数可以表示  $N$  轮内可信度，可疑分数的范围是 0~1。

---

**算法 2：基于模型一致性的客户端评分算法**

---

**输入：** 全局模型差别  $\Delta W_t = \{\Delta w_{t-N}, \Delta w_{t-N+1}, \dots, \Delta w_{t-1}\}$ , 全局模型更新差别  $\Delta G_t = \{\Delta g_{t-N}, \Delta g_{t-N+1}, \dots, \Delta g_{t-1}\}$ , 客户端数  $n$ , 迭代轮次  $n_{\text{epoch}}$

**输出：** 可疑分数

```

1 for  $t = 1, 2, \dots, n_{\text{epoch}}$  do
2    $\hat{H}_t = L - \text{BFGS}(\Delta W_t, \Delta G_t)$ 
3   for  $i = 1, 2, \dots, n$  do
4      $\hat{g}_i^t = g_i^{t-1} + \hat{H}_t(w_t - w_{t-1})$ 
5   end for
6    $d^t = [\|\hat{g}_1^t - g_1^t\|_2, \|\hat{g}_2^t - g_2^t\|_2, \dots, \|\hat{g}_n^t - g_n^t\|_2]$ 
7    $\hat{d}^t = \frac{d^t}{\|d^t\|_1}$ 
8    $s_i^t = \frac{1}{N} \sum_{r=0}^{N-1} \hat{d}_i^{t-r}$ 

```

---

显而易见的，如果一个平常可信的客户端突然出现了一个反常梯度，会通过公式 x 极大拉高未来 N 轮的可疑分数，而经过 N 轮，才能消除该轮次反常梯度造成的分数影响；反之一个平常不可信的客户端突然出现了一个似乎可信的梯度，也会拉低未来 N 轮的可疑分数。

### 3.2.4 恶意门槛选取

恶意门槛是本章算法的核心参数，如果其值设置很高，虽然更能保证安全性，但由于其过滤掉了许多可能蕴含非常见信息或是处于模型剧烈变化阶段导致可疑分数较高的梯度更新，会导致模型拟合更慢且无法学习到一些偶发情况；

如果其值设置较低，会导致攻击者提交的恶意梯度更容易突破防御，对模型造成更多破坏，得不偿失；

如果恶意门槛值设置比较合理，就能充分切合本章算法的提出的几点原则，既能有效屏蔽绝大多数异常梯度，又能谨慎低限度学习一些非常见信息。对于个别呈现“临界规避”行为的恶意客户端，由于其非常贴合正常梯度，造成的破坏有限，且能通过后续其他手段进一步进行限制，最大程度保证安全。对于一些在临界点浮动的蕴含非常见信息的客户端，也能通过可疑分数的逐渐下降最终越过恶意门槛进行再次检验判断。

由于以上原因，依据实际情况设置合适的恶意门槛是有必要的。

文章最终选取恶意门槛的值为 $s_{th}=0.01$ ，其依据主要有两方面：其一，论文需要和前人的优秀模型进行对比，而 FLDetector 文中作者在类似实验环境下分析时，采信的分界线为 $s_{th}=0.01$ ，因此为了控制划分标准一致以进行比对，本文采用相同的标准；其二，通过后续实验输出的可疑分数变化与真实客户端设置情况进行比对，确认 $s_{th}=0.01$  是一个比较合理的分割值。

算法整体流程如算法 2 所示，加入该算法的整体训练流程如图 3-3。

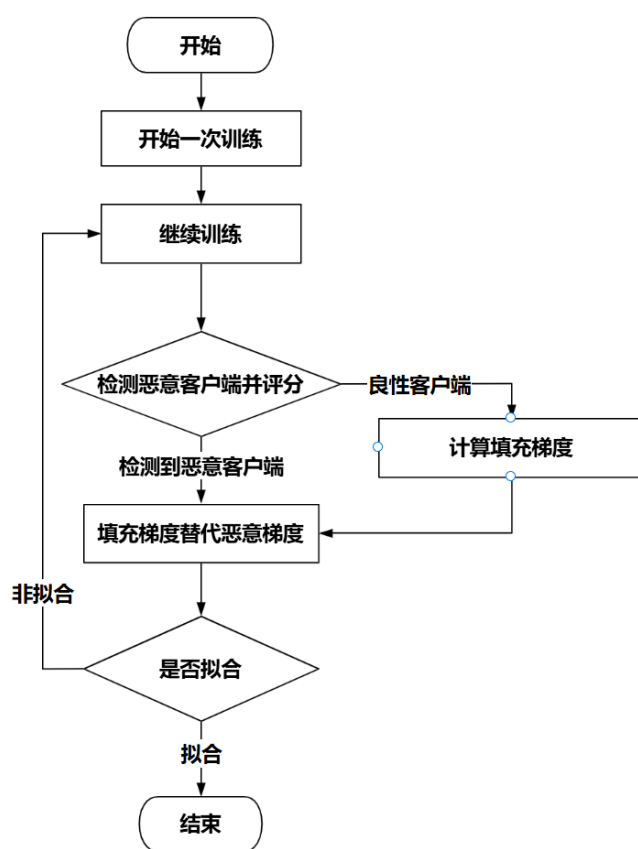


图 3-3 基于模型一致性的检测评分机制训练流程图

Fig 3-3 Training process of detection scoring mechanism based on model consistency

使用均等权重的算法的 Mean-fill 模型运作流程如算法 3 所示。

---

**算法 3: Mean-fill**

---

**输入:** 全局模型差别  $\Delta W_t = \{\Delta w_{t-N}, \Delta w_{t-N+1}, \dots, \Delta w_{t-1}\}$ , 全局模型更新差别  $\Delta G_t = \{\Delta g_{t-N}, \Delta g_{t-N+1}, \dots, \Delta g_{t-1}\}$ , 客户端数  $n$ , 聚合方法 aggregate, 计算 tacc 方法 evaluateTacc, 恶意阈值  $s_{th}$ , 学习率  $\alpha$ , 迭代轮次  $n_{epoch}$

**输出:** 测试准确度

```

1 for t = 1, 2, ..., nepoch do
2    $\hat{H}_t = L - \text{BFGS}(\Delta W_t, \Delta G_t)$ 
3   for i = 1, 2, ..., n do
4      $\hat{g}_i^t = g_i^{t-1} + \hat{H}_t(w_t - w_{t-1})$ 
5   end for
6    $d^t = [\|\hat{g}_1^t - g_1^t\|_2, \|\hat{g}_2^t - g_2^t\|_2, \dots, \|\hat{g}_n^t - g_n^t\|_2]$ 
7    $\hat{a}^t = \frac{d^t}{\|\hat{a}^t\|_1}$ 
8    $s_i^t = \frac{1}{N} \sum_{r=0}^{N-1} \hat{a}_i^{t-r}$ 
9   根据分数和阈值的大小, 判断可疑程度, cnt = 0
10  for i = 1, 2, ..., n do
11    if  $s_i^t > s_{th}$  then
12      判断为恶意客户端, 跳过 i 客户端更新
13    else if  $s_i^t < s_{th}$  then
14      判断为可信客户端
15      val = val +  $g_i^t$ 
16      cnt = cnt + 1
17    end if
18  end for
19  mval = val / cnt
20  for i = 1, 2, ..., n do
21    if  $s_i^t > s_{th}$  then
22       $g_i^t = mval$ 
23    end if
24  end for
25   $g^t = \text{aggregate}(g_1^t, g_2^t, \dots, g_n^t)$ 
26   $w_{t+1} = w_t - \alpha g^t$ 
27  tacc = evaluateTacc( $w_{t+1}$ )
28  记录 tacc
29 end for
30 return 本轮所有 tacc
```

---

### 3.3 本章小结

本章提出了一个针对高 Non-IID 下恶意客户端模型攻击识别问题的新思路。模型将识别过程转换成梯度更新一致性预测过程, 基于 Hessian 矩阵的特性利用其

拟牛顿法的近似能力来模拟客户端模型的更新迭代过程，从而判断其提交的真实模型更新是否合理，此外，为了降低 Hessian 矩阵的计算消耗，还使用 L-BFGS 进行进一步近似。由于该方法只依赖于单个客户端的拟合过程，所以没有传统的检测方法那样受制于数据集 Non-IID 的缺点，同时，用 L-BFGS 优化之后的算法，其对设备算力的要求大大降低，使得其能更好适用于联邦学习的应用场景。

## 4. 基于可疑分数的动态处理梯度算法

实际联邦学习的场景中各个客户端的环境较为复杂，存在一定的误判可能，不能简单的把本轮所有暂时可信的客户端都等同处理。本章在结合前文的检测评分机制的基础上，对经过判分之后的客户端，依据可信度进行进一步处理，从而在尽力保证最大程度安全的基础上，提高汲取信息的质量和效率，并给予蕴含非常见信息的客户端一定的参与权。

### 4.1 均等权重的缺陷和临界规避行为

在联邦学习中，客户端上传的梯度不仅携带了模型更新的信息，也可能暗含部分敏感数据。为了提高防御的鲁棒性，一种思路是在聚合阶段对那些刚好“越过”检测门槛的梯度进行动态加权，而不是简单地对所有客户端等权平均。该动态加权机制基于每个客户端的可疑分数与设定阈值之间的差值，能够更精细地反映各客户端的相对可信度。举例来说，如果客户端 A 的可疑分数远高于防御门槛，而客户端 B 仅略微超过门槛，则简单平均法会将二者赋予相同的权重，而动态加权则会根据分值差异赋予 A 较高的权重，从而更好地抑制那些采用边缘策略（即“临界规避”，即攻击者试图将本地更新控制在刚好超过检测门槛的水平，以逃避检测）的恶意客户端。

已有研究（例如 Melis 等人<sup>[34]</sup>在探讨协同学习中的梯度泄露问题时指出，攻击者可能通过巧妙调整更新幅度使得其贡献恰好处于防御阈值附近）表明，对于采用“临界规避”策略的攻击者，动态加权能利用连续的可信度分布信息，赋予可信客户端更高的权重，从而在聚合阶段削弱那些边缘恶意更新对全局模型的影响。同时，从参数更新的角度来看，动态加权生成的聚合梯度本质上是各可信客户端梯度的一个非均匀凸组合，这一组合更好地保留了各个本地更新的几何分布特性；相比之下，简单平均法隐含了各梯度等重要性的假设，当客户端数据存在较大分布差异或梯度方向分散时，等权平均容易使有效信息被稀释或失真。

此外,从对抗鲁棒性的角度出发,动态加权机制构建了类似双重防御的屏障:一方面,通过对可疑分数与门槛差值进行非线性映射(例如采用 Sigmoid 或 ReLU 等函数),动态加权能够对那些仅刚好超过门槛的梯度给予较低权重;另一方面,在实际聚合过程中,该机制使得一旦部分客户端的更新幅度仅微弱超标,其贡献在全局更新中所占的比例自然降低,从而迫使攻击者必须提交明显高于正常水平的更新才能影响全局模型,从而大大提高了攻击代价。

总的来说,基于可疑分数差值的动态加权机制相较于简单平均具有以下优点:

(1) 对抗临界规避效果更好<sup>[64,65]</sup>:动态加权利用连续可信度信息,可以有效抑制那些仅勉强超过防御门槛的恶意更新,从而提高聚合操作的鲁棒性和安全性。

(2) 更好地提取重要信息<sup>[66]</sup>:动态加权构造出的聚合梯度作为可信客户端梯度的非均匀凸组合,更能保留各自局部更新在参数空间中的方向信息,尤其在 Non-IID 数据场景下,相比等权平均处理能减少由于梯度方向发散带来的失真问题。

虽然具体的数值结果需要通过严谨的实验验证,但已有工作的趋势表明,在面对自适应攻击(如梯度反转攻击)时,动态加权机制能够在模型收敛速度和最终准确性上显示出优于简单平均的优势。

## 4.2 基于可疑分数的动态处理梯度算法

本节将介绍基于可疑分数和恶意门槛的动态聚合算法,首先介绍算法的指导思想,其次介绍算法大概流程,并在上一章模型基础上进一步对比并分析各个算法的优劣。

### 4.2.1 算法流程概述

本章提出的算法系统架构如图 4-1 所示,流程如算法 4 所示,其流程是在前一章计算出每个客户端本轮的可疑分数基础上,引入一个合适的恶意门槛,基于恶意门槛分割善恶客户端,利用良性客户端与恶意门槛的差值作为权重,依据公式构造本轮填充梯度以代替恶意客户端本轮参与聚合的梯度更新,然后提交给后

续选定的聚合算法进行处理。

---

**算法 4:** Weighted-fill

---

**输入:** 全局模型差别  $\Delta W_t = \{\Delta w_{t-N}, \Delta w_{t-N+1}, \dots, \Delta w_{t-1}\}$ , 全局模型更新差别  $\Delta G_t = \{\Delta g_{t-N}, \Delta g_{t-N+1}, \dots, \Delta g_{t-1}\}$ , 客户端数  $n$ , 聚合方法 aggregate, 计算 tacc 方法 evaluateTacc, 恶意门槛  $s_{th}$ , 学习率  $\alpha$ , 迭代轮次  $n_{epoch}$

**输出:** 测试准确度

```

1 for t = 1, 2, ..., nepoch do
2    $\hat{H}_t = L - \text{BFGS}(\Delta W_t, \Delta G_t)$ 
3   for i = 1, 2, ..., n do
4      $\hat{g}_i^t = g_i^{t-1} + \hat{H}_t(w_t - w_{t-1})$ 
5   end for
6    $d^t = [\|\hat{g}_1^t - g_1^t\|_2, \|\hat{g}_2^t - g_2^t\|_2, \dots, \|\hat{g}_n^t - g_n^t\|_2]$ 
7    $\hat{d}^t = \frac{d^t}{\|d^t\|_1}$ 
8    $s_i^t = \frac{1}{N} \sum_{r=0}^{N-1} \hat{d}_i^{t-r}$ 
9   根据分数和门槛的大小, 判断可疑程度, cnt = 0
10  for i = 1, 2, ..., n do
11    if  $s_i^t > s_{th}$  then
12      判断为恶意客户端, 跳过 i 客户端更新
13    else if  $s_i^t < s_{th}$  then
14      判断为可信客户端,
15       $w_i^t = s_{th} - s_i^t$  为可信权重
16       $val = val + w_i^t * g_i^t$ 
17      cnt = cnt + 1
18    end if
19  end for
20   $mval = val / cnt$ 
21  for i = 1, 2, ..., n do
22    if  $s_i^t > s_{th}$  then
23       $g_i^t = mval$ 
24    end if
25  end for
26   $g^t = \text{aggregate}(g_1^t, g_2^t, \dots, g_n^t)$ 
27   $w_{t+1} = w_t - \alpha g^t$ 
28  tacc = evaluateTacc( $w_{t+1}$ )
29  记录 tacc
30 end for
31 return 本轮所有 tacc

```

---

该算法的思想主要可以归纳成以下三点原则:

1. 屏蔽恶意客户端: 对确认为恶意的客户端, 暂时屏蔽其更新;

2.谨慎采纳低可信客户端：对可信度较低的客户端，少量采纳其信息，降低潜在风险；

3.充分采纳高可信客户端：对可信度较高的客户端，赋予更高的权重，充分采纳其信息。

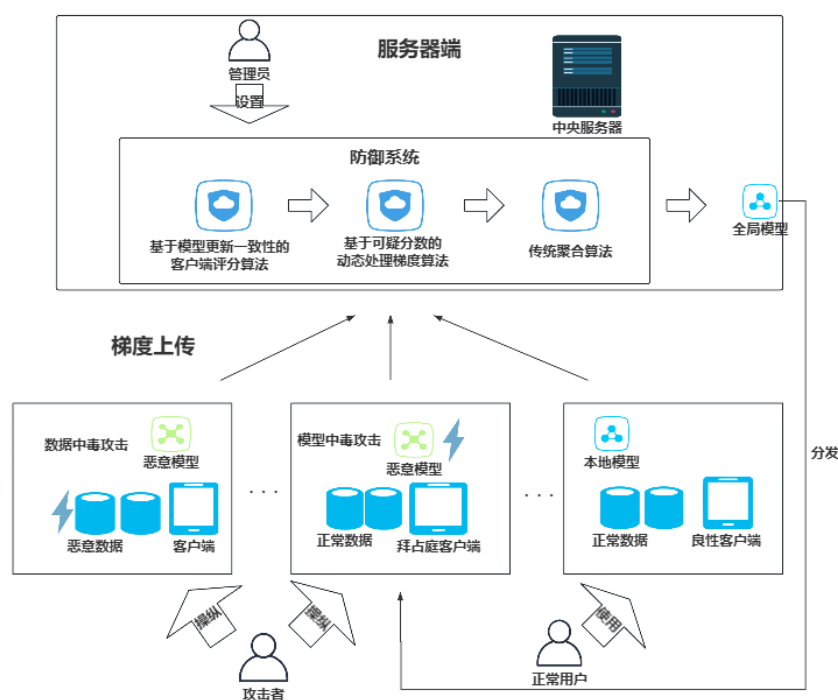


图 4-1 动态处理算法系统架构图  
Fig 4-1 The system architecture of dynamic processing algorithm.

#### 4.2.2 算法对可疑分数的影响分析

在第三章已经给出了可疑分数的计算过程、分数随训练的变化过程与分数范围 0~1，并在上一节引入了恶意门槛的设定。

对于第三节末尾提到的平常可信的客户端突然出现了一个反常梯度与平常不可信的客户端突然出现了一个似乎可信的梯度导致分数浮动的情况，文章认为由于门槛 $s_{th}=0.01$ ，所以可信区间为(0,0.01)，不可信区间为[0.01,1)，所以良性客户端因为异常行为被标记为恶意客户端是更容易的，而恶意客户端因为异常行为被

标记为良性客户端是更困难的，由此能一定程度规避攻击者偶尔伪装成正常客户端进行刷分的情形。

例如，某良性客户端 A 的上一轮的最终恶意评分为 0.005，意味着过去 N 轮均分为 0.005，简单假设  $N=10$ ，过去 10 轮分数都是 0.005。新一轮提交反常梯度，造成原始分数（未结合最近 N 轮平均的分数）波动大小为 0.1，分摊到包含本轮次的 10 轮上，那么 A 的本轮恶意评分越过了恶意门槛到达 0.015。

恶意客户端 B 的上一轮的恶意评分为 0.03，简单假设  $N=10$ ，过去 10 轮分数都是 0.03。假设极限情况，本轮次攻击者伪装造成原始可疑分数接近于 0+，那么造成的波动是 0.03，均摊到包含本轮次的 10 轮上最终结果本轮恶意评分是 0.027，仍然分数非常高。

恶意客户端 C 的上一轮的恶意评分为 0.015，简单假设  $N=10$ ，过去 10 轮分数都是 0.015。假设极限情况，本轮次攻击者伪装造成原始可疑分数接近于 0+，那么造成的波动是 0.015，均摊到包含本轮次的 10 轮上最终结果本轮恶意评分是 0.0135，仍然在可疑区间。

摇摆客户端 D 的在上一轮的恶意评分为 0.0101 极为贴近门槛，简单假设  $N=10$ ，过去 10 轮分数都是 0.0101。假设极限情况，本轮次原始可疑分数接近于 0+，那么造成的原始分数波动为约为 0.01，那么本轮最终恶意评分为 0.00909，堪堪越过门槛，但是采信权重很低。

因为门槛的设置，单轮次下极限良性方向波动造成的影响最多是  $0.01/(N-1)$ （保证前方  $N-1$  轮都要越过门槛），而恶意方向波动的影响则大得多，所以攻击者要伪装成正常客户端进行刷分是很困难的，至少短期内不行。而如果攻击者放任客户端正常训练并伺机进行攻击的话，平时大量轮次在辅助全局模型训练，如果出现剧烈攻击或者连续的普通程度攻击，会导致可疑分数陡升，导致客户端被长期屏蔽；如果是偶尔较温和的攻击，会导致其分数上涨造成采信权重低，结合后续的其他防御措施，再加上攻击频率低，攻击者很难达到他想要的效果。

对于一些游离于恶意门槛周围的客户端，根据分数浮动，低限度采信其信息，以学习到一些特殊信息。

### 4.3 本章小结

本章提出了一种基于可疑分数的动态处理梯度策略，旨在解决联邦学习场景中客户端环境复杂、误判风险高等挑战。本章先介绍了传统均等权重造成的"临界规避"问题，通过引入动态加权机制与多层级防御策略，在提升安全性的同时更高效汲取有价值的信息。首先，算法采用可疑分数与阈值差值的动态加权策略，细粒度区分客户端可信度，其次，算法设定动态恶意门槛，结合历史分数滑动平均机制平衡安全性与信息利用率。最后，分析推导了该策略如何与前文可疑分数算法一起，规避攻击者可能存在的刷分行为与更好的采集摇摆客户端的信息。

## 5. 基于滑动窗口的中值滤波聚合算法

在联邦学习场景中，恶意客户端可能通过全局模型中分开注入隐蔽的恶意触发器从而实现特定目的。前文提出的算法是针对非目标攻击进行防御设计，并没有直接考虑目标攻击手段。本章提出一种中值滤波聚合算法，旨在让整体模型对一些目的修改特定参数的目标攻击也有更好的抵抗能力。

### 5.1 传统聚合算法缺陷

传统的常用有鲁棒性的聚合算法一般包括：Median，Trimmed Mean，Krum，Bulyan<sup>[67]</sup>等。

Bulyan 可以看成 Krum 的简单变换，Krum 需计算所有客户端间的梯度相似性，其计算复杂度为  $O(n^2)$ ， $n$  为客户端数量，这就导致了其虽然能提供不错的鲁棒性，但是难以在大规模的联邦学习中应用；其次，Krum 的本质思想是计算某梯度和其他所有梯度的相似性，从而保证就算选中了恶意梯度，因为其和大部分良性梯度相似，破坏力也有限，但是这里隐含一个假设就是大部分的客户端是良性的，如果恶意客户端的比例达到较高的程度，那么很容易造成恶意梯度之间的相似导致恶意梯度被频繁选中；除此以外，前人已经证明，可以通过反转梯度的思想设计出针对于 Krum 的中间值梯度，这样该梯度很有可能被 Krum 选中。

Median 对每个参数维度独立计算中位数，复杂度是  $O(n \log n)$ 。但忽略了参数之间的相关性，如果攻击者在多个相关参数上同时注入小的触发器，Median 很有可能无法单独捕捉这种微小的异常，从而导致这些相关参数上的小触发器组合起来形成后门攻击绕过防御。Trimmed Mean 的缺点和 Median 非常类似，也是无法检测一些小的扰动。

图像处理中卷积层经常使用到滑动窗口的技术，并在窗口内进行特征提取或者处理；在信号处理领域，也会用低通滤波来去除一些脉冲噪声<sup>[68]</sup>，通过这些滑动窗口处理思想能有效抑制“孤立”极端值对最终聚合的影响。

通过把中值滤波<sup>[69]</sup>的思想引入到联邦学习的聚合函数领域内，文章考虑把传统的 Median 函数进行维度上的拓展，从原来的参数向量拓展形成多参数向量组成的滑动窗口，在该窗口内进行处理。

其系统架构图如图 5-1 所示，其在前一章的基础上把传统的聚合算法替换成了本章提出的聚合算法。

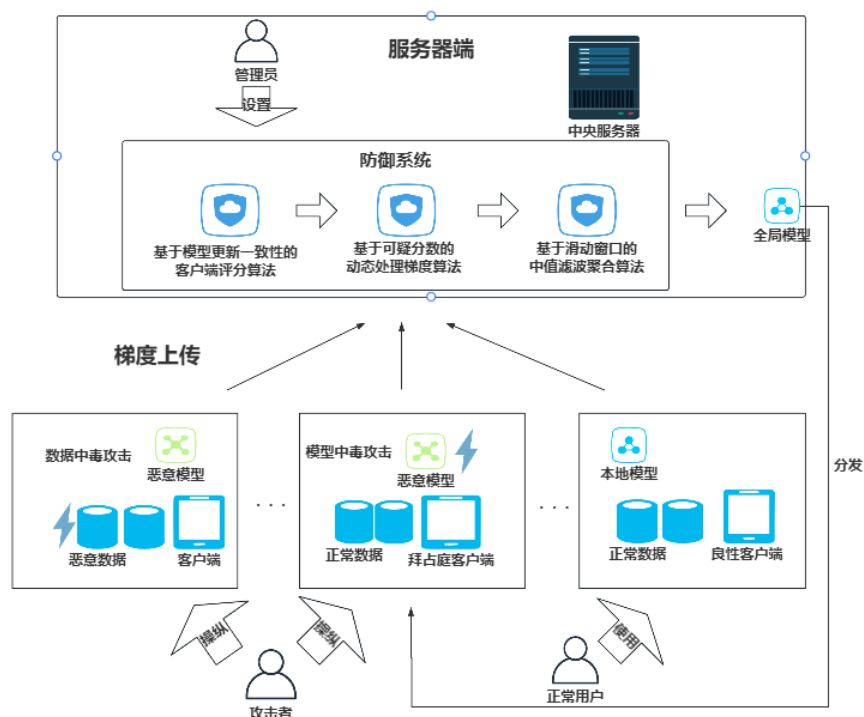


图 5-1 中值滤波聚合算法系统架构图  
Fig 5-1 The system architecture of median smoothing filter algorithm.

## 5.2 中值滤波聚合算法

### 5.2.1 卷积的参数相关性

在联邦学习中，客户端模型提交给中央服务器的参数是其本地模型的参数训练结果，可以看成是该本地模型各层神经元权重的组合。

在卷积神经网络中，每个卷积核通常是一个  $3 \times 3$  或者  $5 \times 5$  的矩阵，专门用来提取输入图的某些特征。而卷积核中相邻的权重，对应的是输入像素矩阵上的相邻的像素位置。权重和像素共同决定了一个区块的输出。

比如卷积核  $\begin{bmatrix} w_{0,0} & w_{0,1} & w_{0,2} \\ w_{1,0} & w_{1,1} & w_{1,2} \\ w_{2,0} & w_{2,1} & w_{2,2} \end{bmatrix}$ , 其中  $w_{1,1}$  和  $w_{1,2}$  是水平相邻的两个权重, 分别

负责提取同一行上相邻像素的特征。因为它们都在“同一个感受野”里工作, 所以在训练过程中, 这些相邻权重往往会学到互补、相似的模式<sup>[70]</sup> (比如共同检测一条边或纹理的一部分), 形成了参数上的相关性<sup>[71]</sup>, 这为共同处理提供了理论依据。

### 5.2.2 滑动窗口

滑动窗口是要求包含目标参数在内的周围一共 `winSize` 个参数进行训练 (个数不足则进行截断), 本质目的是稀释异常值, 从而使得攻击者操纵参数的代价变得极高。

例子: 一共 5 个客户端, 所有客户端模型提交的正常梯度为  $w = [-1, 0, 1, -2, 0, 2, -1, 0, 1]$ , 攻击者想篡改第 5 个参数  $w[4]$  从 0 改成 5, 也就是恶意客户端提交  $w^* = [-1, 0, 1, -2, 5, 2, -1, 0, 1]$ , 正常来说  $w[4]$  的中值是 0。

如果是 Median 算法, 并且恶意客户端小于 50%, 比如 2 个, 那么  $w[4]$  参数的值向量就是  $[0, 0, 0, 5, 5]$ , 能够过滤掉恶意值。如果超过 50%, 比如 3 个, 那么  $w[4]$  参数的值向量就是  $[0, 0, 5, 5, 5]$ , 导致恶意值被选中, 攻击生效。

如果是利用滑动窗口联合周边参数, 对  $w[4]$  取窗口为 3,  $w[4]$  参数的值向量就是  $[-2, -2, -2, -2, -2, 0, 0, 2, 2, 2, 2, 2, 5, 5, 5]$ , 中值是 2。虽然有偏差, 但是相对于 Median 的完全失效被操纵的情况, 我们的算法在大多数客户端都被控制的情况下仍然有不小的抵抗力。

### 5.2.3 算法流程概述

算法流程如算法 5 所示, 该算法接收上游传输的一组客户端梯度向量, 输出聚合后的全局模型参数向量。

1. 引入一个滑动窗口长度的变量 `winSize` (默认 3, 必须为奇数), 其代表以当前参数为中心, 一共多少参数参与聚合 (本身+左边  $winSize/2$ +右边  $winSize/2$  个,

如果不足就截断)。

2.计算一个参数的聚合结果时,对所有参与参数对应的进行 L2 归一化并记录当前参数的归一化尺度。

3.对归一化之后的所有参与数值进行取中位数。

4.对中位数用第二步记录的归一化尺度复原,作为当前参数的聚合结果,依次类推到其他参数。

---

**算法 5:** 基于滑动窗口的中值滤波聚合算法

---

**输入:** 客户端数量  $n$ , 参数个数  $q$ , 一组客户端梯度  $g_1^t \sim g_n^t$ , 窗口大小  $winSize$

**输出:** 聚合结果向量

```

1 将各客户端提交的梯度组合成矩阵  $M[1..q, 1..n]$ 
2 for  $i = 1, 2 \dots q$ :
3   for  $j = i - winSize, \dots, i, \dots, i + winSize$ :
4     如果  $j < 1$  或者  $j > q$  则 continue
5     将当前参数向量  $M[j, 1..n]$  进行 L2 归一化, 并加入集合  $S$ 
6   end for
7 对集合  $S$  的数据取中值得到  $ans'[j]$ 
8 对  $ans'[j]$  按照参数  $j$  的归一化缩小倍数反归一化得到  $ans[j]$ 
9 end for
10 return  $ans$ 
```

---

#### 5.2.4 算法优势分析

1.时间复杂度分析: 归一化部分复杂度是  $O(n)$ , 计算中值的复杂度是  $O(3 * n \log n)$ , 反归一化的复杂度是  $O(n)$ , 最后单个参数的计算复杂度和 Median 一样是  $O(n \log n)$ , 总体来说复杂度并不高。

2.对特定参数攻击的防御能力和对攻击者的阻碍能力: 对于一些需要实现精密操作以实现特殊目标的目标攻击, 需要篡改关键的几个参数而非广泛影响全部的参数。由于高烈度的攻击容易被检测, 攻击者很可能将参数扰动分解在少数几个相关联的参数维度上, 这样 Median 和 Trimmed Mean 算法检测微小的扰动会变得更困难, 在高恶意客户端比例的情况下甚至被操纵, 具体在上一节已经阐述。

而本章的方法通过联动周围参数从而引入更多正常值, 将一些异常值挤出序列的中间, 迫使攻击者如果想要达到以前的攻击效果必须额外控制窗口中相邻的

其它参数维度或者将控制客户端提高几倍才行。

而在目标场景下，如果随意改动额外的参数维度，不仅会使得异常更容易被发现(恶意参数数量扩大，偏向于无目标攻击，破绽更大，容易被前文的检测算法捕捉到)，而且很有可能破坏攻击者精心控制的训练走向，虽然会使训练有一些偏离，但是攻击者并不能达成其特殊目的；

控制更多的客户端对攻击者来说更是麻烦的事，比如算法窗口取 3，意味着相似的参数量大了 3 倍，而恶意参数被稀释了，更有可能离开中间位置，如果攻击者希望达到和 Median 中一样的影响力，则需要控制 3 倍以上的客户端，这对许多攻击者来说是非常困难的。这些问题无疑大大提高了攻击成本与难度。

### 5.2.5 算法潜在缺点分析

1.如果攻击者采用广泛进行参数修改的攻击比如各种非目标攻击，其设计的恶意梯度并不需要像后门攻击那样在关键参数间实现巧妙的关联，攻击者可以广泛的改变所有的参数从而破坏训练。对于这种情况，本章提出的中值滤波聚合算法并不能起到特别出色的作用，因为任意一个参数其相邻的维度都可能被攻击者随意影响，这样算法就退化为 Median。

2.窗口更适合在一组关联参数之间滑动（比如一个卷积核权重的内部），由于客户端提交的梯度是对所有模块参数展平拼接而成，所以会出现窗口跨越不同的卷积核或者其他模块的情况，在这种情况下，如果取到其他模块的参数，可能对模型学习造成负面影响。后续的改进可以是根据具体的客户端模型的参数构成，确定具体的窗口滑动的区域，避免出现跨界。

3.窗口的大小需要具体确定：如果窗口太大，可能引入很多不相干参数，从而让聚合结果更有可能取到完全不相关的参数，对模型性能不利；如果窗口太小，攻击者更容易在该区间内施以强大影响力，导致防御力不足。

## 5.3 本章小结

本章基于窗口化和中值滤波思想提出一种中值滤波聚合函数，旨在应对一些

目标攻击场景下的攻击，其运用滑动窗口技术在不多的额外时间复杂度花费下引入更多数据对恶意值进行稀释，迫使攻击者要么控制多几倍的客户端要么广泛更改参数，从而提高了攻击者的进攻成本与难度，最后简单分析其可能存在的缺点，指出其可能存在跨模块影响拟合和窗口大小需要具体界定的问题。

## 6. 实验设置与结果分析

### 6.1 基础实验设置

#### 6.1.1 防御者的知识与能力

防御措施在中央服务器上运行，接触不到客户端上的原始训练数据集，不知道具体哪些客户端是恶意的，不知道恶意客户端的数量。防御方服务器训练中唯一能接触到的是每轮各个客户端提交上来的本地更新以及中央的全局模型。

#### 6.1.2 攻击者的知识与能力

攻击者可以往网络注入假的客户端，也可以攻破并控制真的客户端，通过数据中毒或者模型中毒发送有毒梯度更新毒害全局模型。攻击者可以操控被控制的客户端的一切数据集和训练过程，但是不能直接影响其他良性客户端和中央服务器。以上叙述的是一般情况下的部分知识攻击(**partial knowledge attack**)，此外还有一种完全知识攻击(**full knowledge attack**)，攻击者也能知道其他良性客户端的训练情况和提交的数据，但是不能操纵和篡改，这种情况并不常见，往往是安全系统出现了极大漏洞，因此一般作为一种极限情况加以参考。对于 Krum 和梯度反转等攻击，前人已经证明在完全知识背景下攻击的强度更大，本文中也考虑了这些攻击。在攻击名称前面加上 **full** 表示是在完全知识情境下的实验，在攻击名称前面加上 **partial** 表示是在部分知识情境下的实验。

#### 6.1.3 攻击类型

中毒攻击可以大致分为目标攻击和非目标攻击，我们这里的攻击类型基本和 FLDetector 模型设置的一致，下面简要介绍一下涉及的几种攻击：

##### 1) 数据中毒攻击(Data poisoning attack,DPA)

攻击者主要通过篡改训练数据集来误导客户端的模型，从而把错误信息更新到全局模型中去，实验中主要涉及到的是标签反转攻击(**Label Flip attack**)，对于数

据集的数据和类别标签统一进行错位,让本地模型学到错误的信息进行提交。

尽管模型更新一致性是针对模型中毒攻击(MPA)的概念,但我们仍然引入DPA来观察模型的鲁棒性。

## 2) 模型中毒攻击 (Model poisoning attack,MPA)

MPA旨在通过直接构建或控制本地模型训练来提交误导性信息,从而破坏全局模型。实验中主要涉及Fang等人<sup>[21]</sup>提出的攻击方法,包括Trimmed Mean攻击、反向梯度攻击(Mean Attack)、高斯攻击(Gaussian Attack)和Krum攻击。除此之外,还包括缩放攻击(Scaling Attack)。

缩放攻击(Scaling Attack):缩放攻击的思想很简单,就是通过放大自身控制服务器所提交的梯度参数来扩大在聚合中的权重影响,由此来达到影响效果。通常有一个缩放因子 $a$ ,该客户端提交的梯度都会乘以 $a$ 。

修剪均值攻击(Trimmed Attack):包括完全知识和部分知识两种情况,是专门针对Trimmed Mean方法的攻击。攻击者用控制客户端的自然更新求均值来近似全局均值,然后让这些本地更新向均值靠近。

反转梯度攻击(Mean Attack):包括完全知识和部分知识两种情况,攻击者对于控制的自然梯度更新取反方向,以达到抵消模型训练的效果。

高斯攻击(Gaussian Attack):计算恶意客户端的均值和方差,并以此生成符合高斯分布的新恶意梯度。

Krum攻击(Krum Attack):包括完全知识和部分知识两种情况。完全知识情况下把攻击转化成优化问题,搜索符合条件的 $\lambda$ 以构造本地梯度更新。部分知识情况下需要用恶意客户端的自然梯度更新去估算良性客户端情况,然后按完全知识情况计算。

后门攻击(Backdoor Attack):通过植入特定触发器,篡改特定参数,使模型在正常数据上表现出高精度,但遇到某些特殊数据,就会输出特定错误结果。

分布式后门攻击(DBA):通过分布式协同将单个后门拆分成多个子任务,从而使攻击更隐蔽,聚合之后形成一个完整的后门触发器。

#### 6.1.4 聚合方法

##### 1) Trimmed mean:

该方法在裁剪后取均值。首先将所有客户端的更新按每个参数分组排序，然后移除最大和最小的  $k$  个异常值，剩下的  $n-2k$  个值取均值作为全局模型参数。

##### 2)Median:

顾名思义，该方法取参数中位数。首先将所有客户端的更新按每个参数分组排序，然后取每组的中位数作为对应的全局模型参数。

##### 3)Krum:

该方法找到一个与其他  $n-k-2$  个更新的欧几里得距离之和最小的本地更新，并将其作为全局更新。其意义在于，即使选择了恶意更新，该更新也足够接近绝大多数更新，从而限制其破坏性。

##### 4)Simple mean(FedAvg):

该方法是最原始的方法，直接取所有梯度的均值作为全局梯度。

##### 5)中值滤波聚合算法 (MedianFilter) :

在第五章中提出的聚合算法。

#### 6.1.5 实验模型设置

Weighted-fill 模型（如图 3-3 和算法 4 所示）会根据良性客户端的可疑分数与恶意门槛之间的差值来确定加权重，从而生成新的填充梯度。这种方法放大了高可信信息的影响，同时削弱了低可信客户端的作用，避免了不可信更新的干扰。

消融模型 Mean-fill（如图 3-3 和算法 3 所示）在检测到恶意更新后，采用良性梯度的均值来替代恶意梯度进行聚合。

FLDetector 对比模型包括两种场景：

FLDetector100 (D100) 为无检测流程、直接依靠聚合算法鲁棒性进行一次性训练，如图 6-1；

而 FLDetector80 (D80) 为标准流程，如图 3-2，即模型通过检测和自监督学习标记恶意客户端，终止训练以移除恶意节点后重新训练。这里假设已多次提前

检测并屏蔽了 28 个中 20 个恶意客户端，仅剩 8 个可能潜伏发动攻击。

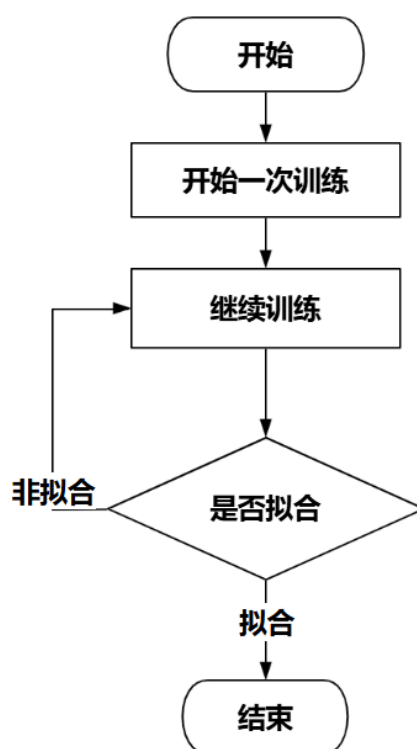


图 6-1 D100 训练流程  
Fig 6-1 Training process of D100

如表 6-1，通过简单分析各个模型的鲁棒性组成，可以更直观对比分析各个模块的作用大小。

表 6-1 各模型的鲁棒性组成  
Table 6-1 Robustness Composition of Models

| 模型            | 鲁棒性来源                 |
|---------------|-----------------------|
| D100          | 聚合函数                  |
| D80           | 聚合函数+检测重启模块           |
| Mean-fill     | 聚合函数+Mean-fill 模块     |
| Weighted-fill | 聚合函数+Weighted-fill 模块 |

#### 6.1.6 联邦学习参数设置

1)客户端数量：假定有  $n$  个客户端，在 D100、Mean-fill 和 Weighted-fill 中以  $n=100$  训练，只有在 D80 模型中以  $n=80$  训练。

2)数据集和分配方式:我们使用两个非常常用的数据集 MNIST 和 CIFAR10。需要设定一个值  $p(0 < p < 1)$  代表 Non-IID 程度,我们设置当  $p = 0.1$  的时候代表 IID,意味着各个客户端上的本地数据集分布几乎一致;  $p$  越高,代表各个客户端上的数据集分布越不相似,越偏向 Non-IID。具体分配方法是假设标签总共有  $k$  类,其中第  $i$  类有  $p$  的概率分配到第  $i$  个客户端,剩下的在其他客户端随机均匀分配。我们的实验设置  $p$  分别为 0.1 和 0.5 分别代表独立同分布和非独立同分布,以测试我们的模型在 IID 和 Non-IID 下的表现。

表 6-2 FCNN 架构设置  
Table 6-2 FCNN architecture

| 层           | 大小      |
|-------------|---------|
| 输入          | 28*28*1 |
| 全连接层 + ReLU | 256     |
| 全连接层 + ReLU | 256     |
| SoftMax     | 10      |

表 6-3 CNN 架构设置  
Table 6-3 CNN architecture

| 层           | 大小      |
|-------------|---------|
| Input       | 28*28*1 |
| 卷积层 + ReLU  | 5*5*30  |
| Max Pooling | 2*2     |
| 卷积层 + ReLU  | 5*5*50  |
| Max Pooling | 2*2     |
| 展平          | -       |
| 全连接层 + ReLU | 512     |
| SoftMax     | 10      |

3)全局模型架构:考虑到 MNIST 比较简单,对于 MNIST,我们采用常用的卷积神经网络 CNN 和全连接神经网络 FCNN 作为全局模型,它们的详细架构如表 6-2 和表 6-3 所示。因为 CIFAR10 比 MNIST 复杂许多,采用 FCNN 的去学习的话,会学习到非常多无用的信息且容易超出内存,适合 MNIST 的 CNN 结构也不能很好提取 CIFAR10 的信息,所以对于 CIFAR10,采用专门处理复杂图像的经典深度残差网络 ResNet20 作为全局模型。

4)联邦聚合算法:我们采用了 5 种聚合算法,包括经典的 FedAvg 中的 Simple

Mean 算法以及 3 种鲁棒的聚合算法 Trimmed Mean、Krum 和 Median，此外还有第五章提出的新聚合算法。

5)训练轮次和学习率:考虑到 CIFAR10 的数据量相对较大,为保证充分训练,训练轮次为 2000 轮,学习率初始设置为  $1 \times 10^{-3}$ ,后期会动态降低学习率。对于 MNIST 数据集,统一标准仍然训练 2000 轮次,学习率设置为  $2 \times 10^{-4}$ 。所有客户端上每梯度下降一次后就提交梯度更新。

6)攻击设置:恶意客户端数量和 FLDetector 保持一致,默认为 28 个,在 D80 模型中以默认已经检测出 20 个,所以以 8 个恶意客户端训练。攻击类型和 3.1 中所述一致。每轮都进行攻击。

7)批大小:默认 batch size 为 32,和 FLDetector 保持一致。

8)恶意门槛:是我们的模型需要用到的门槛参数,默认  $s_{th} = 0.01$ 。

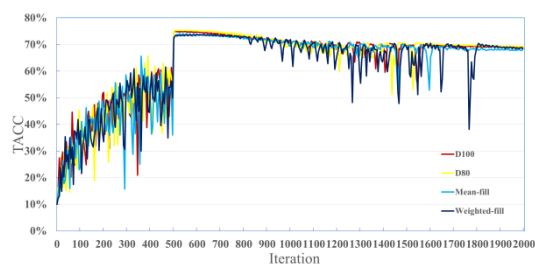
9)评估指标:我们的目标主要是展示出模型相对原模型的鲁棒性,所以在相同的外部条件下对比几个模型的测试准确度( TACC)以及曲线稳定情况来评估鲁棒性。

10)检测机制启动:检测机制有暖启动过程,在 MNIST 上训练 50 轮后启动检测,在 CIFAR10 上 20 轮之后启动。

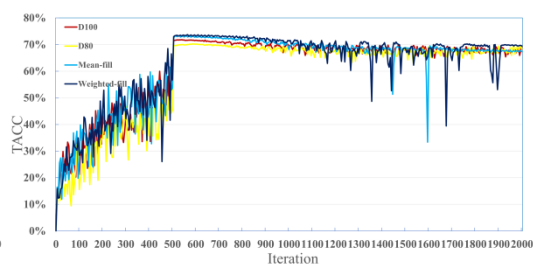
11)实验系统信息如表 6-4 所示。

表 6-4 实验环境  
Table 6-4 Experimental environment

| 设备或软件     | 详细信息                                     |
|-----------|------------------------------------------|
| 操作系统      | Ubuntu20.04.2                            |
| 内存        | 500GB                                    |
| 处理器       | Intel(R) Xeon(R) Gold 6330 CPU @ 2.00GHz |
| 显卡        | NVIDIA GeForce RTX 3090                  |
| CUDA 版本   | 11.8                                     |
| Python 版本 | 3.10.14                                  |
| Mxnet 版本  | mxnet-cu113 1.9.1                        |

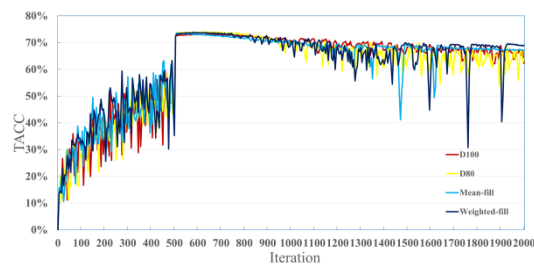


(a)No attack + Simple mean 场景下的 TACC;



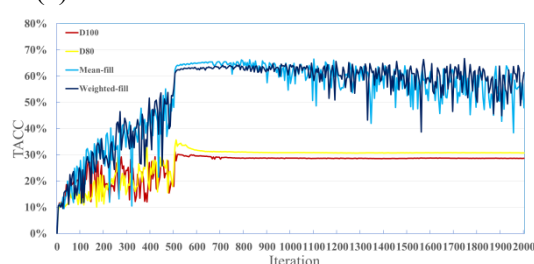
(b)No attack + Median 场景的 TACC;

(a)TACC under No attack + Simple mean scenario; (b) TACC under No attack + Median scenario;



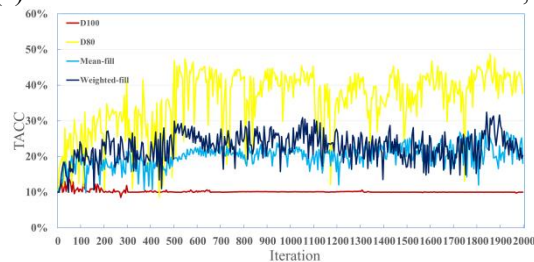
(c)No attack + Trimmed mean 场景的 TACC;

(c)TACC under No attack+Trim mean scenario;



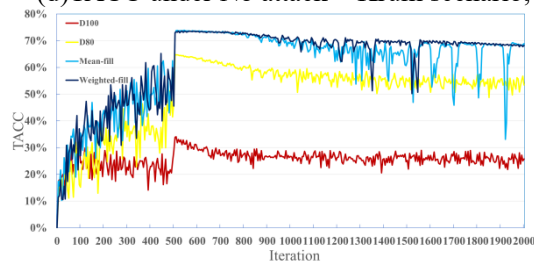
(d)No attack + Krum 场景的 TACC;

(d)TACC under No attack + Krum scenario;



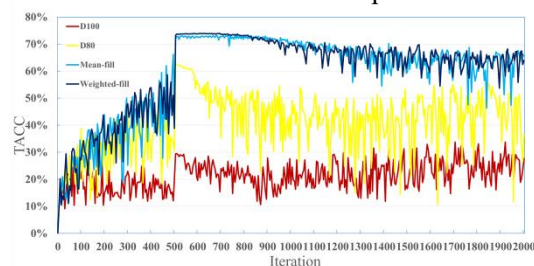
(e)Partial trim attack+Simple mean 场景的 TACC;

(e)TACC under Partial trim attack+Simple mean scenario;



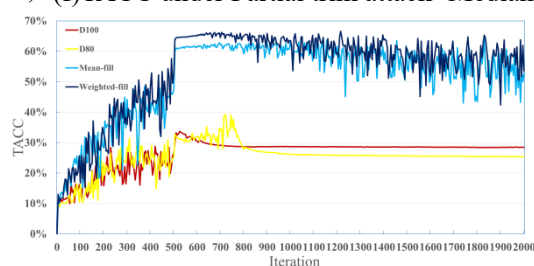
(f)Partial trim attack+Median 场景的 TACC;

(f)TACC under Partial trim attack+Median scenario;



(g)Partial trim attack + Trimmed mean 场景的 TACC;

(g)TACC under Partial trim attack+Trim mean scenario;

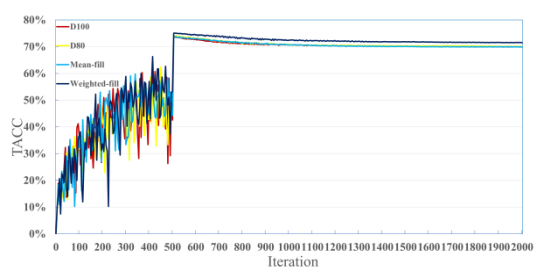


(h)Partial trim attack + Krum 场景的 TACC;

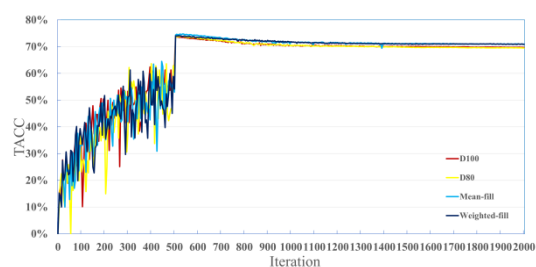
(h)TACC under Partial trim attack+Krum scenario;

图 6-2 Non-IID+CIFAR10 下攻击前后不同聚合函数的 TACC 对比

Fig 6-2 TACC comparison of aggregation functions before/after attack (IID+CIFAR10)

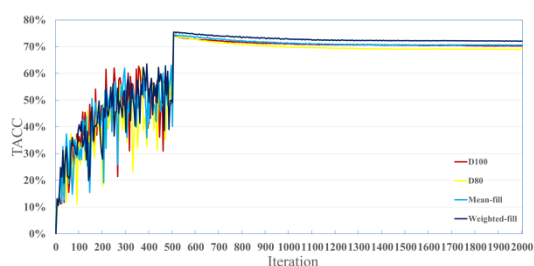


(a)No attack + Simple mean 场景下的 TACC;

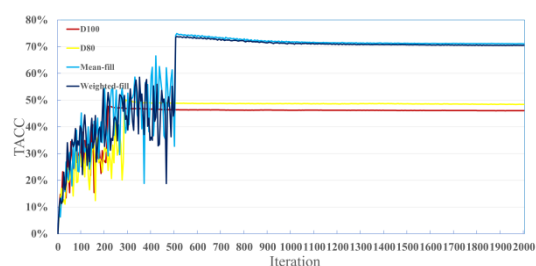


(b)No attack + Median 场景的 TACC;

(a)TACC under No attack + Simple mean scenario; (b) TACC under No attack + Median scenario;



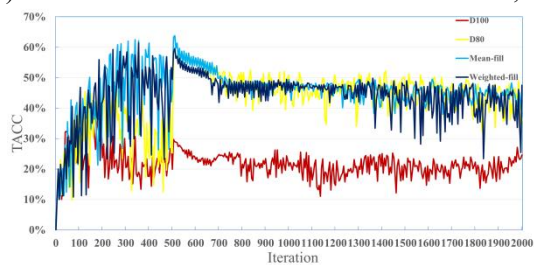
(c)No attack + Trimmed mean 场景的 TACC;



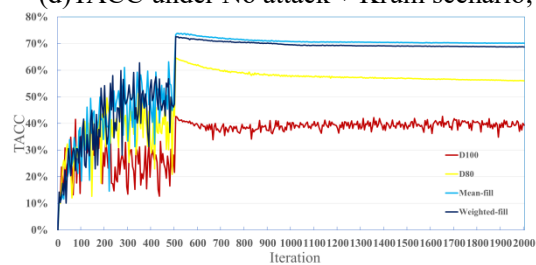
(d)No attack + Krum 场景的 TACC;

(c)TACC under No attack+Trim mean scenario;

(d)TACC under No attack + Krum scenario;

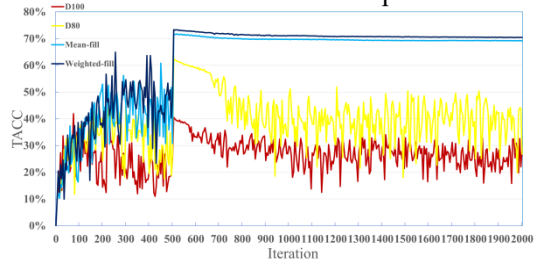


(e)Partial trim attack+Simple mean 场景的 TACC;

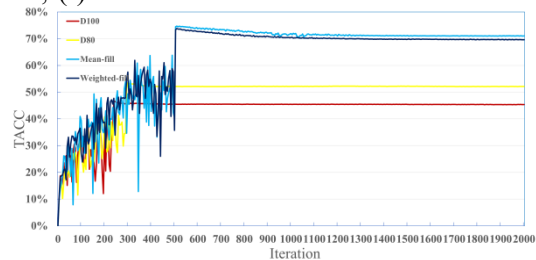


(f)Partial trim attack+Median 场景的 TACC;

(e)TACC under Partial trim attack+Simple mean scenario; (f)TACC under Partial trim attack+Median scenario;



(g)Partial trim attack + Trimmed mean 场景的 TACC;



(h)Partial trim attack + Krum 场景的 TACC;

(g)TACC under Partial trim attack+Trim mean scenario; (h)TACC under Partial trim attack+Krum scenario;

图 6-3 IID+CIFAR10 下攻击前后不同聚合函数的 TACC 对比

Fig 6-3 TACC comparison of aggregation functions before/after attack (IID+CIFAR10)

## 6.2 可疑分数评分模块与动态处理梯度模块的实验

本部分主要对第三章和第四章的算法进行实验对比，具体细节如下文叙述。

表 6-5 模型在 CIFAR10 数据集上结合不同聚合函数的测准度

Table 6-5 TACC (%) of models on CIFAR10 Dataset with different aggregation algorithms

| Cifar-0.5 + Resnet20 |             |       |               |                   |                      |       |               |                   |
|----------------------|-------------|-------|---------------|-------------------|----------------------|-------|---------------|-------------------|
| 聚合                   | 无攻击 TACC(%) |       |               |                   | 部分知识 Trim 攻击 TACC(%) |       |               |                   |
|                      | D100        | D80   | Mean<br>-fill | Weighted<br>-fill | D100                 | D80   | Mean<br>-fill | Weighted<br>-fill |
| Median               | 71.82       | 70.23 | 73.27         | 73.69             | 34.06                | 64.81 | 73.89         | 73.61             |
| Simple               |             |       |               |                   |                      |       |               |                   |
| Mean                 | 74.9        | 75.34 | 74.09         | 73.8              | 13.57                | 48.78 | 27.22         | 32.4              |
| Krum                 | 31.17       | 35.7  | 66.48         | 66.7              | 33.67                | 39.21 | 63.99         | 66.55             |
| Trim                 | 73.15       | 73.95 | 73.59         | 73.9              | 33.74                | 62.34 | 73.39         | 73.14             |
| Cifar-0.1 + Resnet20 |             |       |               |                   |                      |       |               |                   |
| 聚合                   | 无攻击 TACC(%) |       |               |                   | 部分知识 Trim 攻击 TACC(%) |       |               |                   |
|                      | D100        | D80   | Mean<br>-fill | Weighted<br>-fill | D100                 | D80   | Mean<br>-fill | Weighted<br>-fill |
| Median               | 73.69       | 73.86 | 74.72         | 74.05             | 42.62                | 64.6  | 73.87         | 72.6              |
| Simple               |             |       |               |                   |                      |       |               |                   |
| Mean                 | 73.8        | 74.07 | 73.81         | 75.07             | 37.26                | 59.04 | 63.84         | 61.41             |
| Krum                 | 47.69       | 50.07 | 74.88         | 73.79             | 46.66                | 53.5  | 74.71         | 73.8              |
| Trim                 | 73.82       | 73.64 | 74.38         | 75.45             | 42.09                | 62.59 | 71.71         | 73.33             |

### 6.2.1 实验分析

表格 6-5~6-10、图 6-2、6-3 以及附录的图表显示了我们的模型和对比模型在 MNIST 和 CIFAR10 上配合多种聚合方法和攻击组合之后的指标组合 TACC。

#### (1) 聚合算法鲁棒性评估：

通过对比相同模型在不同聚合算法下有无攻击时的 TACC（测试准确率）变化情况来评估鲁棒性。从表 6-5 和表 6-6 观察同一个模型不同聚合算法在无攻击和有攻击之间的性能差值，可以得出以下结论：

(a) FedAvg 并没有鲁棒性，在无攻击下性能不错，但一旦遭受攻击，会有巨大性能损失，在表格 6-5 中展示了训练受攻击影响的情况导致 TACC 很低的情况；

表 6-6 模型在 CIFAR10 数据集上在不同攻击场景的测准度  
Table 6-6 TACC (%) of models on CIFAR10 dataset in different attack scenarios

| Cifar-0.5 + Median + Resnet20 |       |       |           |               |
|-------------------------------|-------|-------|-----------|---------------|
| 攻击                            | 模型    |       |           |               |
|                               | D100  | D80   | Mean-fill | Weighted-fill |
| No                            | 71.82 | 70.23 | 73.27     | 73.69         |
| Partial Trim                  | 34.06 | 64.81 | 73.89     | 73.61         |
| Full Trim                     | 45.74 | 69.74 | 53.59     | 50.24         |
| Partial Mean                  | 57.62 | 69.69 | 63.13     | 61.81         |
| Full Mean                     | 47.46 | 69.59 | 50.75     | 52.07         |
| Gaussian                      | 72.19 | 70.98 | 71.92     | 71.23         |
| Dir Partial Krum              | 35.22 | 66.95 | 65.12     | 64.67         |
| Dir Full Krum                 | 60.43 | 69.6  | 63.47     | 64.8          |
| Label Flip                    | 73.64 | 71.07 | 72.1      | 73.89         |
| Scaling                       | 67.9  | 69.83 | 72.43     | 72.5          |
| Cifar-0.1 + Median + Resnet20 |       |       |           |               |
| 攻击                            | 模型    |       |           |               |
|                               | D100  | D80   | Mean-fill | Weighted-fill |
| No                            | 73.69 | 73.86 | 74.72     | 74.05         |
| Partial Trim                  | 42.62 | 65.17 | 73.87     | 72.6          |
| Full Trim                     | 51.38 | 70.6  | 49.51     | 51.91         |
| Partial Mean                  | 55.94 | 69.93 | 65.84     | 64.32         |
| Full Mean                     | 49.67 | 70.6  | 49.54     | 51.18         |
| Gaussian                      | 70.81 | 73.18 | 71.43     | 71.34         |
| Dir Partial Krum              | 52.34 | 70.27 | 70.64     | 71.53         |
| Dir Full Krum                 | 68.12 | 72.59 | 55.6      | 65.85         |
| Label Flip                    | 74.72 | 73.77 | 71.59     | 71.34         |
| Scaling                       | 71.64 | 69.66 | 73.27     | 73.22         |

(b)Median 和 Trimmed Mean 的性能代价不高，能抵御一些攻击，提供一定的鲁棒性，但是防御效果有限；

(c)从表 6-5 中可以看到，无论无攻击还是 Partial Trim 攻击下，Krum 的性能变化都不大，体现出其鲁棒优势。同时，在无攻击情景下，Krum 的 TACC 普遍低于 Median 和 Simple Mean，说明其为提升鲁棒性牺牲了部分性能。然而，引入我们的模型后，Krum 在单次训练中的性能损耗显著降低，在 MNIST 场景下甚至与 Median 或 FedAvg (Simple Mean) 表现相当。而在 CIFAR10 中，其他聚合方法在攻击下 TACC 下降超过 30%，相比之下 Krum 几乎保持不变，且与我们的机制配合后性能开销进一步减少。

表 6-7 模型在 MNIST 数据集上配合 CNN 模型在不同聚合函数上的测准度  
Table 6-7 TACC (%) of models on MNIST dataset and CNN with aggregation algorithms

| Mnist-0.5 + Cnn |             |       |           |                   |                      |       |           |                   |
|-----------------|-------------|-------|-----------|-------------------|----------------------|-------|-----------|-------------------|
| 聚合              | 无攻击 TACC(%) |       |           |                   | 部分知识 Trim 攻击 TACC(%) |       |           |                   |
|                 | D100        | D80   | Mean-fill | Weighted<br>-fill | D100                 | D80   | Mean-fill | Weighted<br>-fill |
| Median          | 98.32       | 98.37 | 96.91     | 98.88             | 96.46                | 98.17 | 98.03     | 98.63             |
| Simple          |             |       |           |                   |                      |       |           |                   |
| Mean            | 98.99       | 98.92 | 99.05     | 98.85             | 88.36                | 98.61 | 98.28     | 98.18             |
| Krum            | 94.51       | 94.18 | 98.98     | 98.9              | 95.44                | 95.25 | 98.97     | 98.95             |
| Trim            | 98.4        | 98.64 | 98.85     | 98.77             | 98.15                | 98.73 | 98.82     | 98.73             |
| Mnist-0.1 + Cnn |             |       |           |                   |                      |       |           |                   |
| 聚合              | 无攻击 TACC(%) |       |           |                   | 部分知识 Trim 攻击 TACC(%) |       |           |                   |
|                 | D100        | D80   | Mean-fill | Weighted<br>-fill | D100                 | D80   | Mean-fill | Weighted<br>-fill |
| Median          | 98.69       | 98.88 | 99.03     | 98.85             | 98.28                | 98.67 | 98.95     | 98.93             |
| Simple          |             |       |           |                   |                      |       |           |                   |
| Mean            | 99.08       | 99.04 | 98.91     | 99.01             | 97.97                | 98.46 | 98.73     | 98.67             |
| Krum            | 94.85       | 94.71 | 99.01     | 98.83             | 94.11                | 94.79 | 98.91     | 98.9              |
| Trim            | 98.77       | 98.95 | 98.94     | 99                | 98.21                | 98.89 | 98.87     | 98.93             |

表 6-8 模型在 MNIST 数据集上配合 FCNN 模型在不同聚合函数上的测准度  
Table 6-8 TACC (%) of models on MNIST dataset and FCNN with aggregation algorithms

| Mnist-0.5 + Fcnn |             |       |           |                   |                      |       |           |                   |
|------------------|-------------|-------|-----------|-------------------|----------------------|-------|-----------|-------------------|
| 聚合               | 无攻击 TACC(%) |       |           |                   | 部分知识 Trim 攻击 TACC(%) |       |           |                   |
|                  | D100        | D80   | Mean-fill | Weighted<br>-fill | D100                 | D80   | Mean-fill | Weighted<br>-fill |
| Median           | 96.29       | 96.41 | 96.61     | 96.46             | 94.32                | 96.19 | 96.3      | 95.84             |
| Simple           |             |       |           |                   |                      |       |           |                   |
| Mean             | 96.91       | 97.44 | 96.69     | 96.79             | 87.08                | 96.53 | 95.01     | 95.11             |
| Krum             | 87.74       | 90.03 | 96.76     | 96.66             | 90.99                | 91.04 | 96.9      | 96.89             |
| Trim             | 96.22       | 96.61 | 96.42     | 96.34             | 95.14                | 96.21 | 96.33     | 96.29             |
| Mnist-0.1 + Fcnn |             |       |           |                   |                      |       |           |                   |
| 聚合               | 无攻击 TACC(%) |       |           |                   | 部分知识 Trim 攻击 TACC(%) |       |           |                   |
|                  | D100        | D80   | Mean-fill | Weighted<br>-fill | D100                 | D80   | Mean-fill | Weighted<br>-fill |
| Median           | 96.85       | 97.12 | 96.68     | 96.8              | 95.69                | 96.92 | 96.74     | 96.73             |
| Simple           |             |       |           |                   |                      |       |           |                   |
| Mean             | 96.91       | 97.41 | 96.79     | 96.86             | 95.23                | 96.42 | 96.24     | 96.17             |
| Krum             | 89.74       | 90.53 | 96.96     | 96.95             | 89.61                | 90.72 | 96.97     | 97                |
| Trim             | 96.86       | 97.21 | 96.84     | 96.81             | 95.6                 | 96.91 | 96.67     | 96.62             |

表 6-9 模型在 MNIST 数据集上配合 CNN 在不同攻击场景的测准度  
Table 6-9 TACC (%) of models on MNIST dataset and CNN in different attack scenarios

| Mnist-0.5 + Median + Cnn |       |       |           |               |
|--------------------------|-------|-------|-----------|---------------|
| 攻击                       | 模型    |       |           |               |
|                          | D100  | D80   | Mean-fill | Weighted-fill |
| No                       | 98.32 | 98.37 | 96.91     | 98.88         |
| Partial Trim             | 96.46 | 98.17 | 98.03     | 98.63         |
| Full Trim                | 97.75 | 98.28 | 98.61     | 98.52         |
| Partial Mean             | 98.26 | 98.62 | 98.76     | 98.74         |
| Full Mean                | 97.9  | 98.13 | 98.63     | 98.31         |
| Gaussian                 | 98.72 | 98.44 | 98.82     | 98.81         |
| Dir Partial Krum         | 97.59 | 98.47 | 98.7      | 98.62         |
| Dir Full Krum            | 98.25 | 98.45 | 98.76     | 98.69         |
| Label Flip               | 97.74 | 98.48 | 98.78     | 98.69         |
| Mnist-0.1 + Median + Cnn |       |       |           |               |
| 攻击                       | 模型    |       |           |               |
|                          | D100  | D80   | Mean-fill | Weighted-fill |
| No                       | 98.69 | 98.88 | 99.03     | 98.85         |
| Partial Trim             | 98.28 | 98.67 | 98.95     | 98.93         |
| Full Trim                | 98.56 | 98.74 | 98.53     | 98.64         |
| Partial Mean             | 98.74 | 98.97 | 98.93     | 98.96         |
| Full Mean                | 98.48 | 98.65 | 98.68     | 98.68         |
| Gaussian                 | 98.76 | 98.66 | 98.82     | 98.91         |
| Dir Partial Krum         | 98.64 | 98.81 | 98.86     | 98.76         |
| Dir Full Krum            | 98.81 | 98.99 | 98.81     | 99.02         |
| Label Flip               | 98.16 | 98.9  | 98.97     | 98.87         |

## (2)模型与聚合算法兼容性评估:

在无攻击情况下, 对比单一模型在不同聚合算法下的训练效果, 可以看出模型与各聚合算法的兼容性。由于 FedAvg 的 Simple Mean 缺乏鲁棒性, 我们采用该算法效果作为衡量基准。从表 6-5 中可以看出, D80 和 D100 模型与鲁棒性最强的 Krum 算法兼容性较差, 因而产生较大性能开销, 而我们的模型 Weighted-fill 及其变体 Mean-fill 与 Krum 配合时的性能损失明显较小。此外, 在 Median 和 Trimmed Mean 场景下, Weighted-fill 和 Mean-fill 的损失也略低。

我们的模型与 Krum 算法匹配良好。原因在于 Krum 在聚合过程中会丢弃大量有用但独特的信息, 尤其在 Non-IID 程度较高时, 各客户端训练差异明显, 导致 Krum 难以准确选出代表全局有效信息的更新。而我们的模型首先剔除恶意更新,

随后利用各节点信息合成新的梯度更新，这些合成梯度不仅数量上能替代多个恶意更新，而且包含了所有良性梯度的部分信息，从而更容易被 Krum 选中，使全局模型能够学习到所有良性客户端的更新信息，进而提升整体性能。

表 6-10 模型在 MNIST 数据集上配合 FCNN 在不同攻击场景的测准度  
Table 6-10 TACC (%) of models on MNIST dataset and FCNN in different attack scenarios

| Mnist-0.5 + Median+Fcnn |       |       |           |               |
|-------------------------|-------|-------|-----------|---------------|
| 攻击                      | 模型    |       |           |               |
|                         | D100  | D80   | Mean-fill | Weighted-fill |
| No                      | 96.29 | 96.41 | 96.61     | 96.46         |
| Partial Trim            | 94.32 | 96.19 | 96.3      | 95.84         |
| Full Trim               | 94.74 | 96.06 | 95.46     | 95.63         |
| Partial Mean            | 95.74 | 96.36 | 96.21     | 96.13         |
| Full Mean               | 94.74 | 96.1  | 95.96     | 95.71         |
| Gaussian                | 96.16 | 96.41 | 96.51     | 95.54         |
| Dir Partial Krum        | 94.89 | 96.24 | 95.8      | 95.78         |
| Dir Full Krum           | 95.18 | 96.26 | 95.87     | 96            |
| Label Flip              | 95.75 | 96.58 | 96.68     | 96.84         |
| Mnist-0.1 + Median+Fcnn |       |       |           |               |
| 攻击                      | 模型    |       |           |               |
|                         | D100  | D80   | Mean-fill | Weighted-fill |
| No                      | 96.85 | 97.12 | 96.68     | 96.8          |
| Partial Trim            | 95.69 | 96.92 | 96.74     | 96.73         |
| Full Trim               | 95.62 | 96.8  | 96.02     | 96.11         |
| Partial Mean            | 96.32 | 96.98 | 95.85     | 96.37         |
| Full Mean               | 95.69 | 96.69 | 95.98     | 95.94         |
| Gaussian                | 96.69 | 97.09 | 96.63     | 96.67         |
| Dir Partial Krum        | 95.67 | 96.9  | 96.73     | 96.68         |
| Dir Full Krum           | 96.42 | 97.01 | 96.79     | 96.73         |
| Label Flip              | 96.46 | 97.21 | 97.11     | 97.14         |

### (3)训练效果评估

通过对比同一攻击环境下的性能表现，可以看出机制对训练的性能损耗/辅助训练效果大小。从表格 6-5、表格 6-7、表格 6-8 中在相同外部条件下横向对比四种模型的 TACC 最佳性能，可以看出在多数时候我们的模型 Weighted-fill 以及变体模型 Mean-fill 都能取得不逊于 D100 和 D80 的性能。考虑到过去的许多声称具有鲁棒性的方法在训练时会牺牲一部分准确度来换取鲁棒性，而我们的新训练机制在提升鲁棒性的同时，对原始训练的负担较小。这说明，我们的方法在保障安

全性的同时，能有效减少信息损失，提升训练效率。

#### (4) Non-IID 影响评估：

(a)首先，从图 6-2、6-3 和表 6-5~表 6-10 来看,不论是在 MNIST 还是 CIFAR10 上，Non-IID 的程度较高( $p=0.5$ )的情况下整体训练曲线的表现都要低于 IID( $p=0.1$ )情况。考虑到数据分布差异导致模型训练更困难，获取有效信息的难度更大，这是正常的现象。

(b)其次，通过表 6-5 在 CIFAR10 数据集对 Non-IID( $p=0.5$ )和 IID( $p=0.1$ )场景进行对比，可以发现 Weighted-fill 和 Mean-fill 的性能波动也是要小于等于 D80 和 D100 的，说明 Weighted-fill 和 Mean-fill 适应不同 Non-IID 的能力更强。

(c)此外，在表 6-5 可以进一步看出，在 Non-IID 的无攻击场景下，Weighted-fill 的性能要略好于 Mean-fill 而 IID 场景则相反，考虑到现实场景中 Non-IID 更多，所以我们认为这方面 Weighted-fill 要略好于 Mean-fill。

#### (5)模型鲁棒性评估

根据表 6-5~表 6-10 中不同模型在攻击下 TACC 的损失以及表 6-1 的鲁棒性组成，可以用来评估模型鲁棒性。从表现上看，D100 的鲁棒性主要依赖于聚合算法，相较于其他模型表现最弱，仅在无攻击或低强度攻击并采用简单聚合时能保持甚至略优的性能；一旦遇到攻击或结合更复杂的聚合方法时，其性能便会大幅下降，说明 D100 只能适应无攻击或极弱攻击场景。

而 D80 可看作标准的 FLDetector 方法，其鲁棒性既依赖聚合算法，也依赖检测重启机制，因此在大多数情况下表现良好。虽然在面对完全知识攻击和梯度反转攻击时，D80 展现出较大优势，但对于部分知识攻击及其他攻击，Weighted-fill 和 Mean-fill 往往能保持更好或相近的鲁棒性。这其中有两个原因：首先，完全知识攻击要求攻击者掌握所有良性客户端的信息，属于极端情况，而在实际应用中更常见的是部分知识攻击；其次，我们的模型是在 FLDetector 基础理论上扩展而来，完全兼容且不干扰其检测机制，同时也可在 D80 检测剔除后用于重新训练，从而进一步凸显出我们的模型相对于 D100 的鲁棒优势。

对于 Mean-fill 和 Weighted-fill 而言，这两种模型在大多数情况下表现相近，

但在某些特殊情形下 Weighted-fill 的优势更为明显，例如在 CIFAR-0.5 的 Simple Mean 攻击情境中，因聚合方法本身无法提供足够鲁棒性，Weighted-fill 表现明显更优；而在 CIFAR-0.1 面对 Dir Full Krum 攻击时，Weighted-fill 也显示出巨大的优势。

### 6.2.2 实验小结

实验结果表明，文章提出的 Weighted-fill 模型和消融模型 Mean-fill 显著优于仅依赖聚合函数鲁棒性的 D100 模型。与 FLDetector 的标准模型（D80）相比，即使在 FLDetector 先检测并屏蔽大多数恶意客户端后再进行训练的设置下，我们的模型也仅在少数极端全知识攻击场景下略逊于 D80，并且训练效率大幅提升（因为 FLDetector 涉及额外的自监督训练和反复重启）。此外，我们的模型基于 FLDetector 的框架，但不会破坏其检测机制，因此能够与 FLDetector 的检测和重启机制兼容。在 FLDetector 的严格应用环境中，我们的模型同样能够借助其提前检测和屏蔽功能，能够实现与 D80 相似的效果，实际对比结果表明，Weighted-fill 模型优于 D100 模型。更重要的是，在不同程度的 Non-IID 环境下，Weighted-fill 保持了更好的鲁棒性，且模型性能波动较小，展现出相比传统鲁棒模型的显著优势。经过多维度的分析和评估，文章认为 Weighted-fill 在综合表现上最佳，其次是 Mean-fill，只有在个别极端情况下稍逊于前者；D80 尽管在整体表现上逊色于 Weighted-fill 和 Mean-fill，但在特定场景下仍表现优异。这些结果验证了我们提出的基于梯度一致性评分和动态处理的鲁棒性增强机制在提高模型防御能力和应对 Non-IID 变化方面的有效性。

## 6.3 中值滤波聚合实验

本文基于上文已经验证过的 Weighted-fill 为基础模型进行实验，在 Non-IID 和 Cifar10 数据集环境下，在无攻击、后门攻击(Backdoor attack)、分布式后门攻击(DBA)、以及一些非目标攻击上进行实验，对比的聚合算法有 Median 和 Krum。

表格 6-11 显示了中值滤波算法在实验环境下与其余各聚合方法的 TACC 对比

结果。其中 MedianFilter 代表文章提出的中值滤波算法，winSize 表示滑动窗口的大小。

表 6-11 中值滤波算法 TACC(%)实验结果  
Table 6-11 TACC(%) of MedianFilter algorithm

| Cifar-0.5 + Resnet |               |       |                             |                             |
|--------------------|---------------|-------|-----------------------------|-----------------------------|
| 攻击                 | Weighted-fill |       |                             |                             |
|                    | Median        | Krum  | Median<br>Filter(winSize=3) | Median<br>Filter(winSize=5) |
| No                 | 73.69         | 66.7  | 71.8                        | 70.58                       |
| Partial Trim       | 73.61         | 66.55 | 71.63                       | 68.48                       |
| Backdoor           | 72.02         | 38.55 | 73.95                       | 75.14                       |
| DBA                | 72.37         | 31.77 | 73.9                        | 73.58                       |

通过对比可以明显看到在无攻击场景下使用新算法模型的性能要略低于使用 Median 算法的模型，但是要远高于使用 Krum 算法的模型，这和文章在第五章的分析是符合的，新算法相比于 Median 确实有更多的性能代价，但是要远远小于 Krum。

在非目标攻击场景下，实验选取了常用的部分知识修剪均值攻击，可以看到窗口大小为 5 的场景下，受攻击导致的性能下降更多一些，窗口为 3 的场景下，其在该攻击下提供的鲁棒性和 Krum 与 Median 聚合算法类似。考虑到这并非文章提出的算法的应用场景，我们认为是正常的。

在第五章中，文章指出中值滤波聚合算法是对部分的目标攻击比如后门攻击的特性进行针对性设计的，所以理论上应当在后门相关场景下有更好的表现。通过观察表格，可以发现文章提出的中值滤波算法在后门攻击(Backdoor attack)和分布式后门攻击(DBA)有更好的表现。尤其是面对 Backdoor attack 时，相比于传统的 Median 算法，其测准度(TACC)有约 2%~3%的提升，在 DBA 上，也有 1.21%~1.53%的提升。而 Krum 算法在后门攻击场景下效果很差，表明其无法抵御该攻击。

综合来看，Median 和 Krum 在面对后门攻击时的性能下降程度，说明 Median 在后门攻击上提供的鲁棒性高于 Krum，而中值滤波算法面对两种后门攻击后性能不降反增，更能说明新算法在该场景下除了提供优越的鲁棒性和弥补了性能代价，还对模型起到了额外的增益效果。

经过分析，我们认为可能是以下两个原因：

(1)窗口化机制可能无意中引入了类似正则化（Regularization）的效果，抑制了模型过拟合，从而提升了在正常场景下的泛化能力。

(2)根据第五章中的分析，窗口化机制使得攻击者要想达到在 Median 中类似的效果，必须额外对周围的其他参数施以强大的影响力以进行配合攻击，这意味着必须花费更高的攻击成本。而在相同的攻击成本下，其攻击效果很可能呈现指数级下降。

最后得出的结论是中值滤波聚合算法在后门攻击场景下具有增强模型鲁棒性和提升泛化性的双重效果。

## 6.4 本章小结

本章主要内容是对实验环境进行说明，并进行实验和结果分析。具体来说，首先对实验环境和参数设置进行说明，包括联邦学习训练参数、攻击列表、聚合函数列表，随后对文中第三、四、五章节提出的方法进行实验和数据整理，证明了模型更新一致性的客户端评分算法相对于对比模型的优越鲁棒性以及适用范围的广泛，证明了基于可疑分数的动态处理梯度算法对于前一章算法在一些极端情况下的补足能力，最后通过对中值滤波聚合算法的实验分析，证明其在后门攻击场景下相对于传统算法的优越性。

## 7. 结论与展望

### 7.1 研究结论

在本文中，论文主要研究了在 Non-IID 下数据分布之间的低规律性导致的无法准确检测并防御非目标恶意攻击的问题与传统的聚合算法未能很好防御后门攻击的问题。本文提出了基于模型更新一致性检测评分算法、基于恶意评分的动态梯度处理算法与中值聚合算法来解决这些问题。

首先论文介绍了联邦学习的现状和应用意义，阐述了现在攻击手段的发展、防御手段的局限性和发展 Non-IID 下防御手段的必要性。除此以外，还阐释了一些相关技术以引出可能解决问题的几种思路，进而介绍具体的设计方法。

为解决高 Non-IID 下面对模型中毒攻击的精确检测问题，提出一种基于模型更新一致性的节点评分机制并加以优化，从而在较低算力代价的基础上避免了数据高 Non-IID 对检测精度的影响。实验证明加入该算法的模型相对于只使用聚合算法的模型与 FLDetector 模型在多数场景下有不同程度的鲁棒性提升。

为进一步解决模型信息汲取的安全和效率问题，提出一种基于恶意评分的动态处理梯度机制，充分利用前文算法，从而最大化兼顾训练信息提取的效率和安全性。实验结果证明加入该算法之后的模型，能基本维持基准模型在各场景下的提升和优势，并且在一些额外的攻击场景下有更好的效果。

为了弥补在目标攻击比如后门攻击方面的不足，论文提出一种基于中值滤波思想的窗口化聚合算法，通过扩大化参数从而稀释潜在的恶意值，从而对攻击者的进攻提出了更严苛的要求。实验证明其对后门攻击及其衍生攻击确实有更佳的抵御能力并且能提供额外的增益效果。

综合来看，论文提出的三种算法都能够在各自的场景下面对不同攻击提供一些抵御能力。如果是面对非目标攻击中的模型中毒攻击的话，可以考虑基于模型更新一致性的节点评分机制和基于恶意评分的动态处理梯度机制，并根据实际情况考虑是否兼容使用 FLDetector 中的检测-删除-重启算法，如果是面对目标攻击

中的后门攻击可以考虑加入中值滤波聚合算法。

## 7.2 研究展望

本文提出的几种鲁棒联邦学习训练方法在各自的目标场景中表现优异，但仍然有许多不足之处需要后人进一步地深入研究，可以研究的点不仅限于以下几点：

（1）未来的研究可以进一步探讨如何提升其在更复杂场景中的适应性，尤其是在极端攻击和更高维度数据下的表现以及如何应对联邦学习场景下设备随时上下线的问题。

（2）未来还可以优化与现有检测系统的协同工作，结合自监督学习或对抗训练等技术增强防御能力。

（3）如何解决第五章中分析的中值滤波的缺陷和代价，亦或者把该思想结合应用在一些新的机制上。

（4）此外，随着联邦学习应用场景的扩展，如何将该方法应用于跨领域问题，如自然语言处理或智能医疗等，也将是一个重要的研究方向。

## 参 考 文 献

- [1] Li L, Fan Y, Tse M, et al. A review of applications in federated learning[J]. Computers & Industrial Engineering, 2020, 149: 106854.
- [2] McMahan B, Moore E, Ramage D, et al. Communication-efficient learning of deep networks from decentralized data[C]//Artificial intelligence and statistics. PMLR, 2017: 1273-1282.
- [3] Albrecht J P. How the GDPR will change the world[J]. Eur. Data Prot. L. Rev., 2016, 2: 287.
- [4] Hsu T M H, Qi H, Brown M. Measuring the effects of non-identical data distribution for federated visual classification. arxiv 2019[J]. arxiv preprint arxiv:1909.06335, 1909.
- [5] Zhai K, Ren Q, Wang J, et al. Byzantine-robust federated learning via credibility assessment on non-IID data. arxiv 2021[J]. arxiv preprint arxiv:2109.02396.
- [6] Li X, Huang K, Yang W, et al. On the convergence of fedavg on non-iid data[J]. arxiv preprint arxiv:1907.02189, 2019.
- [7] Paulik M, Seigel M, Mason H, et al. Federated evaluation and tuning for on-device personalization: System design & applications[J]. arxiv preprint arxiv:2102.08503, 2021.
- [8] Federated Learning: Collaborative Machine Learning without Centralized Training[EB/OL].[2025-02-08].<https://research.google/blog/federated-learning-collaborative-machine-learning-without-centralized-training-data/>.
- [9] Lee Y, Cho Y, Han W, et al. Flguard: Byzantine-robust federated learning via ensemble of contrastive models[C]//European Symposium on Research in Computer Security. Cham: Springer Nature Switzerland, 2023: 65-84.
- [10] Vyas A, Lin P C, Hwang R H, et al. Privacy-Preserving Federated Learning for Intrusion Detection in IoT Environments: A Survey[J]. IEEE Access, 2024.
- [11] Lamport L, Shostak R, Pease M. The Byzantine generals problem[M]//Concurrency: the works of leslie lamport. 2019: 203-226.

- [12] Blanchard P, El Mhamdi E M, Guerraoui R, et al. Machine learning with adversaries: Byzantine tolerant gradient descent[J]. Advances in neural information processing systems, 2017, 30.
- [13] Chen Y, Su L, Xu J. Distributed statistical machine learning in adversarial settings: Byzantine gradient descent[J]. Proceedings of the ACM on Measurement and Analysis of Computing Systems, 2017, 1(2): 1-25.
- [14] Awan K A, Din I U, Zareei M, et al. Securing IoT with deep federated learning: A trust-based malicious node identification approach[J]. IEEE Access, 2023, 11: 58901-58914.
- [15] Li T, Sahu A K, Talwalkar A, Smith V. Federated Learning: Challenges, Methods, and Future Directions[J]. IEEE Signal Processing Magazine, 2020, 37(3): 50–60.
- [16] Richardson A, Filos-Ratsikas A, Faltings B. Rewarding high-quality data via influence functions[J]. arxiv preprint arxiv:1908.11598, 2019.
- [17] Li T, Sanjabi M, Beirami A, et al. Fair resource allocation in federated learning[J]. arxiv preprint arxiv:1905.10497, 2019.
- [18] Huang W, Li T, Wang D, et al. Fairness and accuracy in federated learning[J]. arxiv preprint arxiv:2012.10069, 2020.
- [19] Baruch G, Baruch M, Goldberg Y. A little is enough: Circumventing defenses for distributed learning[J]. Advances in Neural Information Processing Systems, 2019, 32.
- [20] Sun Z, Kairouz P, Suresh A T, et al. Can you really backdoor federated learning?[J]. arxiv preprint arxiv:1911.07963, 2019.
- [21] Fang M, Cao X, Jia J, et al. Local model poisoning attacks to {Byzantine-Robust} federated learning[C]//29th USENIX security symposium (USENIX Security 20). 2020: 1605-1622.
- [22] Yin D, Chen Y, Kannan R, et al. Byzantine-robust distributed learning: Towards optimal statistical rates[C]//International conference on machine learning. Pmlr, 2018: 5650-5659.
- [23] Yang Q, Liu Y, Chen T, et al. Federated machine learning: Concept and applications[J]. ACM Transactions on Intelligent Systems and Technology (TIST),

2019, 10(2): 1-19.

- [24] Sattler F, Müller K R, Wiegand T, et al. On the byzantine robustness of clustered federated learning[C]//ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2020: 8861-8865.
- [25] Xie C, Koyejo S, Gupta I. Zeno: Distributed stochastic gradient descent with suspicion-based fault-tolerance[C]//International Conference on Machine Learning. PMLR, 2019: 6893-6901.
- [26] Gupta N, Liu S, Vaidya N H. Byzantine fault-tolerant distributed machine learning using stochastic gradient descent (sgd) and norm-based comparative gradient elimination (cge)[J]. arxiv preprint arxiv:2008.04699, 2020.
- [27] Zhang J, Li C, Robles-Kelly A, et al. Hierarchically fair federated learning[J]. arXiv preprint arXiv:2004.10386, 2020.
- [28] Sim R H L, Zhang Y, Chan M C, et al. Collaborative machine learning with incentive-aware model rewards[C]//International conference on machine learning. PMLR, 2020: 8927-8936.
- [29] Cao X, Fang M, Liu J, et al. Fltrust: Byzantine-robust federated learning via trust bootstrapping[J]. arXiv preprint arXiv:2012.13995, 2020.
- [30] Zhang Z, Cao X, Jia J, et al. Fldetector: Defending federated learning against model poisoning attacks via detecting malicious clients[C]//Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining. 2022: 2545-2555.
- [31] Zhao Y, Li M, Lai L, et al. Federated learning with non-iid data[J]. arXiv preprint arXiv:1806.00582, 2018.
- [32] Mohri M, Sivek G, Suresh A T. Agnostic federated learning[C]//International conference on machine learning. PMLR, 2019: 4615-4625.
- [33] Song Q, Lei S, Sun W, et al. Adaptive federated learning for digital twin driven industrial Internet of Things[C]//2021 IEEE Wireless Communications and Networking Conference (WCNC). IEEE, 2021: 1-6.
- [34] Melis L, Song C, De Cristofaro E, et al. Exploiting unintended feature leakage in collaborative learning[C]//2019 IEEE symposium on security and privacy (SP).

- IEEE, 2019: 691-706.
- [35] Cao X, Jia J, Gong N Z. Provably secure federated learning against malicious clients[C]//Proceedings of the AAAI conference on artificial intelligence. 2021, 35(8): 6885-6893.
  - [36] Liu J, Wu J, Chen J, et al. FedDWA: Personalized federated learning with dynamic weight adjustment[J]. arxiv preprint arxiv:2305.06124, 2023.
  - [37] Ranaweera K, Neiat A G, Liu X, et al. Enhancing Federated Learning Through Secure Cluster-Weighted Client Aggregation[J]. arxiv preprint arxiv:2503.22971, 2025.
  - [38] Wang L, Wang Z, Karimireddy S P, et al. Fedeba+: Towards fair and effective federated learning via entropy-based model[J]. arxiv preprint arxiv:2301.12407, 2023.
  - [39] Wang H, Kaplan Z, Niu D, et al. Optimizing federated learning on non-iid data with reinforcement learning[C]//IEEE INFOCOM 2020-IEEE Conference on computer communications. IEEE, 2020: 1698-1707.
  - [40] Gadekallu T R, Manoj M K, Kumar N, et al. Blockchain-based attack detection on machine learning algorithms for IoT-based e-health applications[J]. IEEE Internet of Things Magazine, 2021, 4(3): 30-33.
  - [41] Zhu J, Cao J, Saxena D, et al. Blockchain-empowered federated learning: Challenges, solutions, and future directions[J]. ACM Computing Surveys, 2023, 55(11): 1-31.
  - [42] Chen J H, Chen M R, Zeng G Q, et al. BDFL: A byzantine-fault-tolerance decentralized federated learning method for autonomous vehicle[J]. IEEE Transactions on Vehicular Technology, 2021, 70(9): 8639-8652.
  - [43] García-Márquez M, Rodríguez-Barroso N, Luzón M, et al. Krum Federated Chain (KFC): Using blockchain to defend against adversarial attacks in Federated Learning[J]. arXiv preprint arXiv:2502.06917, 2025.
  - [44] Nguyen D H, Nguyen P L, Nguyen T T, et al. FedBlock: A Blockchain Approach to Federated Learning against Backdoor Attacks[C]//2024 IEEE International Conference on Big Data (BigData). IEEE, 2024: 7981-7990.
  - [45] Qu Y, Uddin M P,

- Gan C, Xiang Y, Gao L, Yearwood J. Blockchain-enabled Federated Learning: A Survey[J]. ACM Computing Surveys, 2023, 55(4): 1–35.
- [45] Qu Y, Uddin M P, Gan C, et al. Blockchain-enabled federated learning: A survey[J]. ACM Computing Surveys, 2022, 55(4): 1-35.
- [46] Hallaji E, Razavi-Far R, Saif M, et al. Decentralized federated learning: A survey on security and privacy[J]. IEEE Transactions on Big Data, 2024, 10(2): 194-213.
- [47] \*\*e C, Koyejo O, Gupta I. Fall of empires: Breaking byzantine-tolerant sgd by inner product manipulation[C]//Uncertainty in Artificial Intelligence. PMLR, 2020: 261-270.
- [48] Bhagoji A N, Chakraborty S, Mittal P, et al. Analyzing federated learning through an adversarial lens[C]//International conference on machine learning. PMLR, 2019: 634-643.
- [49] Wang H, Sreenivasan K, Rajput S, et al. Attack of the tails: Yes, you really can backdoor federated learning[J]. Advances in neural information processing systems, 2020, 33: 16070-16084.
- [50] Bagdasaryan E, Veit A, Hua Y, et al. How to backdoor federated learning[C]//International conference on artificial intelligence and statistics. PMLR, 2020: 2938-2948.
- [51] Li Y, Jiang Y, Li Z, et al. Backdoor learning: A survey[J]. IEEE transactions on neural networks and learning systems, 2022, 35(1): 5-22.
- [52] Gao Y, Doan B G, Zhang Z, et al. Backdoor attacks and countermeasures on deep learning: A comprehensive review[J]. arXiv preprint arXiv:2007.10760, 2020.
- [53] Xie C, Huang K, Chen P Y, et al. Dba: Distributed backdoor attacks against federated learning[C]//International conference on learning representations. 2019.
- [54] Tolpegin V, Truex S, Gursoy M E, et al. Data poisoning attacks against federated learning systems[C]//Computer security—ESORICs 2020: 25th European symposium on research in computer security, ESORICs 2020, guildford, UK, September 14–18, 2020, proceedings, part i 25. Springer International Publishing, 2020: 480-501.
- [55] Shejwalkar V, Houmansadr A. Manipulating the byzantine: Optimizing model

- poisoning attacks and defenses for federated learning[C]//NDSS. 2021.
- [56] Deng L. The mnist database of handwritten digit images for machine learning research [best of the web][J]. IEEE signal processing magazine, 2012, 29(6): 141-142.
- [57] Krizhevsky A, Hinton G. Learning multiple layers of features from tiny images[J]. 2009.
- [58] Xiao H, Rasul K, Vollgraf R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms[J]. arXiv preprint arXiv:1708.07747, 2017.
- [59] Cohen G, Afshar S, Tapson J, et al. EMNIST: Extending MNIST to handwritten letters[C]//2017 international joint conference on neural networks (IJCNN). IEEE, 2017: 2921-2926.
- [60] Krizhevsky A, Sutskever I, Hinton G E. ImageNet classification with deep convolutional neural networks[J]. Communications of the ACM, 2017, 60(6): 84-90.
- [61] Liu Y, Chang S, Liu Y, et al. FairFed: Improving Fairness and Efficiency of Contribution Evaluation in Federated Learning via Cooperative Shapley Value[C]//IEEE INFOCOM 2024-IEEE Conference on Computer Communications. IEEE, 2024: 621-630.
- [62] Stich S U. Local SGD converges fast and communicates little[J]. arxiv preprint arxiv:1805.09767, 2018.
- [63] Gower R M, Loizou N, Qian X, et al. SGD: General analysis and improved rates[C]//International conference on machine learning. PMLR, 2019: 5200-5209.
- [64] Zhou Y, Zhao L, Matsuo I B M, et al. A dynamic weighted aggregation equivalent modeling approach for the DFIG wind farm considering the Weibull distribution for fault analysis[J]. IEEE Transactions on Industry Applications, 2019, 55(6): 5514-5523.
- [65] Shabanikia N, Nia A A, Tabesh A, et al. Weighted dynamic aggregation modeling of induction machine-based wind farms[J]. IEEE Transactions on Sustainable Energy, 2021, 12(3): 1604-1614.

- [66] Shabanikia N, Khajehoddin S A. Weighted dynamic aggregation modeling of grid-following inverters to analyze renewable DG integrated microgrids[J]. IEEE Transactions on Industrial Electronics, 2023, 71(1): 583-594.
- [67] Guerraoui R, Rouault S. The hidden vulnerability of distributed learning in byzantium[C]//International conference on machine learning. PMLR, 2018: 3521-3530.
- [68] Yin L, Yang R, Gabbouj M, et al. Weighted median filters: a tutorial[J]. IEEE Transactions on circuits and systems II: analog and digital signal processing, 1996, 43(3): 157-192.
- [69] Narendra P M. A separable median filter for image noise smoothing[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1981 (1): 20-29.
- [70] Li D, Li L, Chen Z, et al. Shift-ConvNets: Small convolutional kernel with large kernel effects[J]. arxiv preprint arxiv:2401.12736, 2024.
- [71] \*\* G, Yi X, Zhang L, et al. How does weight correlation affect generalisation ability of deep neural networks?[J]. Advances in Neural Information Processing Systems, 2020, 33: 21346-21356.